



JAVA JUICE

(Core Java interview questions and answers)

-Based on JDK 1.8-

From

jThread

by

Jeeva Rachel Abraham

(Technical Analyst, jThread IT Training & Consultancy)

Thank you my mentor Saju Pappachen Sir who taught me this technology.

I thank the Almighty God for his sustained grace showered on me throughout the course of writing this book.

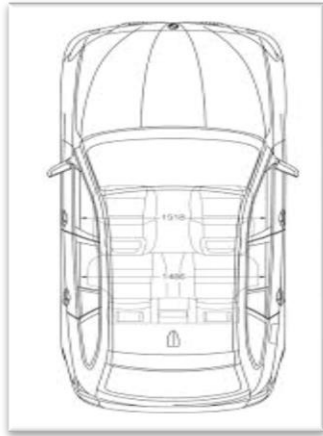
TOPICS

1. JAVA BASICS
2. EXCEPTION HANDLING
3. INPUT/OUTPUT
4. MULTITHREADING
5. COLLECTION FRAMEWORK
6. JDBC
7. DESIGN PATTERNS

JAVA BASICS

1. What is a class in java?

A class is a group of similar objects which have common properties. It is a template or a blueprint from which objects are created. It is not a real one. We can say that class is a picture of an object which comes into our mind.



Class can also be defined as a user defined data type.

A class in java contains:

- ❖ fields
- ❖ methods
- ❖ constructors
- ❖ blocks

Syntax to declare a class

```
class <class name>{  
  
fields;  
  
methods;  
  
}
```

2. What is an object?

Object is a real instance of a class. Each object has its own states and behaviors.



STATE

- name
- age
- height

BEHAVIOUR

- talk
- walk
- eat



STATE

- name
- color
- breed

BEHAVIOUR

- bark
- run
- eat

3. What is new keyword in java?

new keyword is used to create an object of the class. It also allocates memory for it in the heap memory.

4. How many types of memory areas are allocated by JVM?

- ❖ Class(Method) Area
- ❖ Heap
- ❖ Stack

- ❖ Program counter register
- ❖ Native method stack

5. What if I write static public void instead of public static void for the main() method?

Program compiles and runs properly. The order of modifiers doesn't matter.

6. What is a Constructor in java?

Constructor is used to initialize an object at the time of creation itself.

7. What are the Rules for defining a constructor?

- ❖ Constructor name should be same as the class name
- ❖ It cannot contain any return type
- ❖ All access modifiers are allowed (private , public, protected, default)
- ❖ It cannot have any non access modifiers (final ,static, abstract, synchronized etc)
- ❖ It can take any number of parameters
- ❖ A constructor can throw exception or we can declare exceptions using throws for a constructor.

8. What is the use of private constructors in Java?

When we use private for a constructor then object for the class can only be created internally within the class, no outside class can create object for this class. Using this we can restrict the caller from creating objects. This will be helpful in writing a singleton class.

9. Can we have a Constructor in an Interface?

No, we cannot have a constructor defined in an Interface.

10. Can we have a Constructor in an abstract class?

Yes, we can have constructor defined in an abstract class.

11. What is constructor chaining in Java?

Constructor chaining is nothing but calling one constructor from another.

this() is used to call the current class constructor and super() is used to call the parent class constructor.

Eg:

```
class A {  
    public A() {  
        System.out.print("A");  
    }  
  
    public A(int i) {  
        System.out.print("C");  
    }  
}  
  
class B extends A {  
    public B() {  
        System.out.print("D");  
    }  
  
    public B(int j) {  
        System.out.print("B");  
    }  
}  
  
public class Test {
```



```

        public static void main(String[] args) {
            B obj = new B(8);
            // AB will be printed
        }
    }

```

By default a constructor will call its super class default constructor. This happens because of the `super()` statement in the constructor. Even if we don't add this compiler will add this statement. If the super class doesn't have a default constructor it will result in a compile time error. We can also call a parameterized constructor of a super class by calling `super()` with correct order, type and number of arguments of the super class constructor.

12. Can we have `this()` and `super()` in the same constructor?

No, we cannot have `this()` and `super()` in a constructor. Also `this()` or `super()` must be the first statement in the constructor.

13. Is it possible to call a sub class constructor from super class constructor?

No. You cannot call a sub class constructor from a super class constructor.

14. Can we write a class with no constructors in it? What will happen during object creation?

Yes, we can have a class with no constructor, when the compiler encounters a class with no constructor then it will automatically create a default constructor for you.

15. Will compiler create the default constructor when we already have a constructor defined in the class ?

No, the compiler will not create the default constructor when we already have a constructor defined.

16. Why constructors cannot be final in Java?

When you set a method as final, then “The method cannot be overridden by any class”, but constructor by JLS (Java Language Specification) definition can’t be overridden. A constructor is not inherited, so there is no need or doesn’t make sense for declaring it as final.

17. Why constructors cannot be static in Java?

When you set a method as static, it means “The Method belong to class and not to any particular object” but a constructor is always invoked with respect to an object, so it makes no sense for a constructor to be static.

18. What is the purpose of default constructor?

Default constructor is used to provide the default values to the object states like 0, null etc. depending on the type.

19. What is parameterized constructor?

A constructor which has arguments is called parameterized constructor.

20. Does constructor return any value?

No.

21. What happens if you keep a return type for a constructor?

Ideally, constructor must not have a return type. By definition, if a method has a return type, it’s not a constructor. It will be treated as a normal method. But compiler gives a warning saying that method has a constructor name.

22. When do we need constructor overloading?

A class having more than one constructor is called constructor overloading. Sometimes there is a need of initializing an object in different ways. This

can be done using constructor overloading. Different constructors can do different work by implementing different line of codes and are called based on the type and no of parameters passed. So the initialization of objects can be done in different ways. i.e fully or partially.

According to the situation, one constructor is called with specific number of parameters among overloaded constructors.

23. Can we define a method with same name of class?

Yes, it is allowed to define a method with same class name. No compile time error and runtime error is raised, but it is not recommended as per coding standards.

24. How compiler and JVM can differentiate constructor and method definitions of both having same class name?

By using return type, if there is a return type it is considered as a method else it is considered as a constructor.

25.What are the various access specifiers for Java classes?

In Java, access specifiers are the keywords used before a class, method or a variable name which defines the access scope. The access specifiers are:

1. public : class,method,field is accessible from anywhere.
2. protected: method, field can be accessed from the all the classes in the same package and also from subclasses in different packages.
3. default: class, method ,field can be accessed only from the same package and not from outside the package.(If you not use any access modifiers, it will be treated as default access modifier.)

4. private: method, field can be accessed from the same class to which they belong. (If a variable is declared as private, we cannot access it from outside the class. So, we use public methods like setters and getters for accessing this variable. Such classes are called fully encapsulated class.)

26. What are instance variables in Java programming language?

- ❖ Instance variable are variables declared within the class body, outside of any method or block, and declared without 'static' keyword.
- ❖ Instance variables have the second highest scope. Instance variables are created when a new class instance is created, and live until the instance is removed from memory.
- ❖ Instance variables are stored in the heap memory.

```
class Test {  
    int a = 20; //instance variable  
    public void demo(){  
        int b=30;//local variable  
    }  
}
```

27.What are local variables in Java programming language?

- ❖ Local variables are variables declared within a method body.
- ❖ Local variables live only as long as the method in which it is declared remains on the stack.
- ❖ Local variables are stored in the stack memory.

28. What are block variables in Java programming language?

- ❖ Block variables are variables declared within a block such as an init block or within a for loop.
- ❖ Block variables live only during the execution of the block and are the shortest living variables.
- ❖ Block variables are stored in the stack memory.

29. What is the default value of the local variables?

The local variables are not initialized to any default value, neither primitives nor object references. It must be initialized explicitly before using it.

30. What will be the initial value of an object reference which is defined as an instance variable?

The object references are all initialized to null in Java.

31. What kind of memory is used for storing instance variables and local variables ?

Local variables are stored in stack whereas instance variables are stored in heap.

32. Why do instance variables have default values whereas local variables don't have any default value ?

Instance variable are loaded into heap, so they are initialized with default values when an instance of a class is created. But local variables, they are stored in stack until they are being used.

33. What is the difference between instance variable and local variable?

Local variables are visible only in the method or block they are declared whereas instance variables can be seen by all methods in the class.

Instance variables are given default values, i.e. null if it's an object reference, 0 if it's an int etc. Local variables don't get default values, and therefore need to be explicitly initialized before using it.

Instance variables are created when a new instance(object) is created in the heap, and they live until the instance is removed. Local variables live as long as their method remains on the stack.

34. What is static keyword in Java?

static is a non access modifier, static can be applied to variable, method, nested class and initialization blocks (static block). Basically, it is used for memory management so that we can make our memory efficiently or save memory. static means class level in java.

35. What is a static variable ?

- ❖ When we declare any variable with static keyword is known as static variable. Static variable is also known as class variable. When we want to refer a common property to all the objects then we use static variable e.g. student name and roll no may be different but student's school name will be same for all the students of a school.
- ❖ Java static variable gets loaded along with the class.
- ❖ All the instances of the class share the same copy of the static variable.
- ❖ A static variable can be accessed directly by calling “<<ClassName>>.<<VariableName>>” without creating instance for the class.

36. What is a static method ?

- ❖ A static method belongs to the class rather than the object. It can be called directly by using the class name
“<<ClassName>>.<<MethodName>>”
- ❖ A static method can access static variables directly and it cannot access non-static variables. Also it can only call a static method directly and it cannot call a non-static method from it. In order to access a non-static variable or a non-static method from a static method, the object of the class is required.

37. What's the purpose of static methods and static variables?

When there is a requirement to share a method or a variable between multiple objects of a class instead of creating separate copies for each object, we use static keyword to make a method or variable shared for all objects.

38. Can we overload static methods in Java?

Yes, you can overload static methods in Java.

39. Can we override static methods of a class?

We cannot override static methods. Static methods belong to a class and not to individual objects and are resolved at the time of compilation (not at runtime). Even if we try to override static method, we will not get a compilation error, nor the impact of overriding when running the code. i.e Those static methods belong to the respective parent and child classes in which they are present.

40. What is the difference between instance variable and static variable?

- ❖ Static variable has the longest scope. They are created when the class is loaded and they survive as long as the class stays loaded in the JVM.

Instance variables are the next most long-lived. They are created when a new instance(object) is created in the heap, and they live until the instance is removed.

- ❖ Static variables are accessed using the class name but in order to access the instance variable an instance of a class is required.

41. Is it possible to declare a static variable in a method?

You can't declare a static variable inside a method, static variable is a class variable and it belongs to the whole class. This means that static keyword can be used only in a 'class scope' i.e. it doesn't have any sense inside methods. And *final* is the only modifier permitted for a local variable.

42. Can we access non-static data member from static method in java?

No, we can't call non-static data member from a static method directly. You need an object of the class to access it.

Eg:

```
class Test {  
    int x = 10;  
    public  
    static void main(String[] args)
```



```

{
    System.out.println(x);
    //compile time error: cannot make a static reference to the non static field
}
}

```

43. Can the static keyword be applied on outer class?

No. But static keyword may be applied for inner class.

44. How is the memory allocated for static variables internally in java?

Memory allocation for a static variable occurs only once when the program runs for the first time. i.e, when a class is loaded, static variables are also loaded which is shared by every objects of the class.

So memory consumption is very less.

For example:

Imagine a class has a and b declared as static and instance variable respectively.

```

public class VariableDemo {
    static int a;
    int b;
}

```

Now if we create 5 objects of `VariableDemo`, 5 instances of b will be created. i.e. 20 bytes of memory.

And 4 bytes memory will be allocated for a, which is shared by all the 5 objects.

45. Why doesn't Java allow overriding of static methods?

The point of method overriding is that you can subclass a class and the objects implementing those subclasses will have different behaviors for the same methods defined in the super class (and overridden in the subclasses). A static method is not associated with any instance of a class so the concept is not applicable.

46. Can you make a class static in Java?

Yes, you can make a nested class as static in Java.

Nested class

The Java programming language allows you to define a static class within another class. Such a class is called a nested class .

```
class Outer{
    static class Inner{
        void doStuff(){
            System.out.println("Doing");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Outer.Inner inner = new Outer.Inner();
        inner.doStuff();
    }
}
```

Doing will be printed.

47. What is a static block ?

We can declare static block with static keyword and opening and closing curly braces. Java static block is executed before the main() method at the class loading time. It is also used to initialize the static data members. Static block belongs to the class level. There are no arguments and return types allowed.

```
public class StaticBlockDemo {  
    static int num;  
    static String str;  
    static {  
        num = 29;  
        str = "static in Java";  
    }  
  
    public static void main(String args[]) {  
        System.out.println("Value of num: " +  
num);  
        System.out.println("Value of str: " +  
str);  
    }  
}
```

Output:

```
Value of num: 29  
Value of str: static in Java
```

48. Can we have multiple static blocks in a class ?

Yes, we can have more than one static block in our code. It will be executed in the same order it is written.

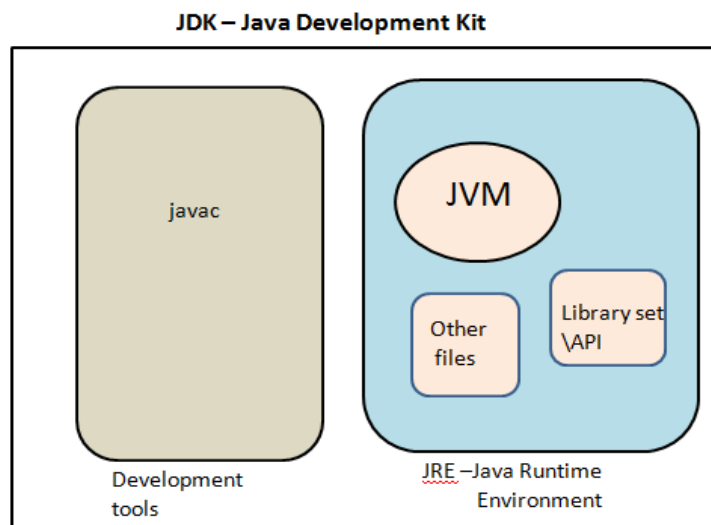
49. Can we access static data member from static method?

Yes, we can access static data member from static method directly.

50. Can we use this and super in static context?

No, because this and super belongs to objects.

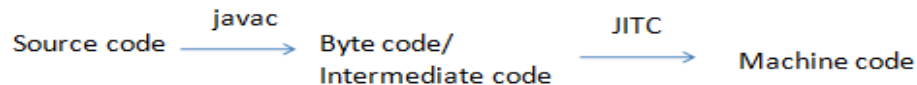
51. Difference between JDK ,JRE and JVM?



- ❖ JDK is the complete package for developing java application. It contains JRE & development tools.
- ❖ JRE provides the environment for the code execution. JRE is the part of JDK. JRE contains JVM, library set and other files..
- ❖ JVM is primarily responsible for running the java application. It is the part of JRE.

52. Why java is called platform independent?

Java is a platform independent programming language. Because when you compile your source code using javac compiler you will get the byte code. This byte code can run on any platform, provided the target platform has the respective JRE installed, i.e JRE for that operating system. When you run the java application JITC does an implicit compilation and converts your byte code into machine code.



53. What is High level language?

High level languages(HLL) are languages that have less complexity and easy to understand, as the syntax is close to human language and it abstracts the machine language.

54. What are the primitive data types in Java ?

There are eight primitive data types.

- ❖ byte
- ❖ short
- ❖ int
- ❖ long
- ❖ float
- ❖ double
- ❖ char
- ❖ boolean

55. Which package is auto imported by default in any java class?

java.lang package and no import statement is necessary.

56. Can I import same package/class twice?

Yes. One can import the same package or same class multiple times. JVM will internally load the class only once no matter how many times one imports the same class.

57. Should package be the first statement in a Java class?

Yes. Otherwise it will result in compilation error. But comments can come before package, but no legal statement can come before package.

58. What is the difference between & and && operator?

&	&&
& is the "bit-wise AND" operator.	&& is the "conditional logical AND" operator.
& always evaluates both arguments.	&& evaluates the first argument. if it is true only, it evaluates the second.

59. What is the difference between ++i and i++ under increment operators? Or what is post and pre increment?

In ++i , the value of i will get incremented before it is used in the expression.

In i++, the previous value is used in the expression first, and after that i is modified and incremented.

```
int i=10;int j=20;int k=30;
System.out.println(i++ + k);//40 =10+30
System.out.println(++j + k);//51 =21+30
```

60. Is there a performance impact due to a large number of import statements that are unused?

No. the unused imports are ignored if the corresponding imported class/package is not used.

61. What is the output?

```
System.out.println(1.0/0);
```

Most of us may expect ArithmeticException, however, in this case, there will be no exception instead it prints Infinity.

1.0 is a double literal and double data type supports infinity.

62. What happens when we forget break in switch case?

The switch statement will continue the execution of all case labels until it finds a break statement, even though those labels don't match the expression value.

63. When no command line arguments are passed, does the String array parameter of main() method be null?

It is an empty array with zero elements and not null.

```
public class MainMethodClass {
    public static void main(String[] args) {
        System.out.println("Size of the Arg
Array="+args.length);
    }
}
```

Output:

Size of the Arg Array=0

64. Difference between `System.exit(0)`, `exit (1)` and `exit(-1)` in Java.

Zero represents that the program ended successfully. Any number greater than zero represents program execution failed. Number less than zero represents program execution error.

65. Can we invoke a static method on a null object reference in Java?

Yes. The compiler optimizes the code to invoke the static method using null object reference since object instance is not required to invoke a static method.

```
public class StaticMethodUsingNull {  
    public static void printSomething(){  
        System.out.println("Hello..");  
    }  
    public static void main(String[] args) {  
        StaticMethodUsingNull s=null;  
        s.printSomething();//print Hello..  
    }  
}
```

In case of calling a non-static (instance) method using null object reference, it will throw `NullPointerException`.

66. What are Loops in Java? What are three types of loops?

Looping is used in programming to execute a statement or a block of statement repeatedly. There are three types of loops in Java:

1) for Loops

for loop is used in java to execute statements repeatedly for a given number of times. For loops are used when number of times to execute the statements is known to the programmer.

2) while Loops

while loop is used when certain statements need to be executed repeatedly until a condition is fulfilled. In while loops, condition is checked first before execution of statements.

3) do While Loops

do While Loop is same as while loop with only difference that condition is checked after execution of block of statements. Hence in case of do while loop, statements are executed at least once.

67. What is an infinite Loop?

An infinite loop runs without any condition and runs infinitely.

68. What is the difference between continue and break statement?

break and continue are two important keywords used in loops. When a break keyword is used in a loop, loop is broken instantly while a continue keyword is used, current iteration is broken and loop continues with next iteration.

69. What is the difference between double and float variables in Java?

In java, float takes 4 bytes in memory while double takes 8 bytes in memory. Float is single precision floating point decimal number while Double is double precision decimal number.

70. What's the base class in Java from which all classes are derived?

java.lang. Object

71. Why java main method is static?

Java main() method is declared as static because JVM has to call this method without creating the object of the class.

Otherwise JVM has to create object first then call main() method that will lead the problem of extra memory allocation.

72. Can main() method in Java can return any data?

In java, main() method can't return any data and hence, it's always declared with a void return type.

73. Can we declare main() method as private or protected or with no access modifier?

No, main() method must be public. You can't define main() method as private or protected or with no access modifier. This is because to make the main() method accessible to JVM. If you define main() method other than public, compilation will be successful but you will get run time error as no main method found.

74. Can we overload main() method?

Yes, we can overload main() method. A Java class can have any number of main() methods. But the rule of method overloading has to be followed.

And in order to run the java program, class should have main() method with signature as “public static void main(String[] args)”. If you do any modification to this signature, compilation will be successful. But, you can’t run the java program. You will get run time error as “Error: Main method not found in class, please define the main method as:

```
public static void main(String[] args)”
```

75. Can we override main method of Java class?

No, we cannot override main method in Java, because main() method is declared as static and static methods cannot be overridden in java. It does not make sense to “override” it, since the concept of “overriding” is only for instance methods.

76. Can we make main method as final in Java ?

Yes, you can make the main() method as final.

77. Can we change return type of main() method ?

No, you cannot change the return type of the main() method, it must be void. If you change the return type then we will get a runtime error “Error: Main method not found in class , please define the main method as:

```
public static void main(String[] args)”
```

78. What are Java Packages?

Package is a set of related class and interface.

Package in java can be categorized in two form, built-in packages and user-defined packages.

There are many built-in packages such as java.lang, java. io, java.util, java.sql etc.

Packages are just similar to folders in the disk.

79. Why packages are used for?

- ❖ Packages are used to organizing the classes and interface.
- ❖ It is used to avoid the naming conflict.

80. What are API in java?

An application programming interface (API), is a collection of prewritten packages having classes, and interfaces that runs on JVM. The inbuilt classes provide functionalities to the developers such that it helps the developers to reduce the number of lines of code.

In order to use a class from java API , we need to import it after the package declaration.

Eg: `import java.util.*;`

Now it will import all the classes in the java.util package.

But the java.lang is automatically imported in a java program and does not need an explicit import statement.

81. Can we declare a class as abstract without having any abstract methods?

Yes, we can declare a class as abstract even if it doesn't have any abstract methods. That means that class is not meant for instantiation.

82. What's the difference between an abstract class and interface in Java?

Interface variables are static and final by default. But abstract class can contain static, non-static, final, non-final variables.

Abstract class can contain abstract and concrete methods. Interfaces also can contain abstract and concrete methods, but the concrete methods must be declared as either static or default.

Interface are implemented by a class using implements keyword whereas abstract class is extended using extends keyword.

Members of interfaces are public by default but abstract class can have private, protected members.

A class can implement multiple interfaces but it can extend only one abstract class.

83. Can we declare a class as protected?

No, but inner classes can be declared as protected.

84. What is method overriding? What is the rule for that?

Changing base class implementation in the subclass is called method overriding.

The key benefit of overriding is that the sub class can provide some different functionality for a method defined in the super class.

Rules:

- ❖ Method signature must be same
- ❖ Argument should be same
- ❖ Return type also should be same
- ❖ Access modifier should be same or more accessible than super class method.

85. What is interface in Java?

Interface is a reference type in java. It is just similar to class. Multiple inheritance cannot be achieved in java using implementation inheritance, i.e class extending another class. To overcome this problem interface are used.

86. Why to use java interfaces?

- ❖ It is used to achieve abstraction.
- ❖ By interface, we can support the functionality of multiple inheritance.
- ❖ It can be used to achieve loose coupling.

87. Can we create an object of an interface?

No, we cannot create an object of interface.

88. Can we declare the interface as final?

No, we can't declare the interface as final because the implementation of the interface is provided by another class. If we make the interface as final, it will result in a compile-time error.

Only public & abstract are permitted for an interface.

89. Which keywords java compiler adds before interface fields and methods?

In an interface, Java compiler adds public, static and final keywords before fields or data members and add public abstract keywords before abstract methods.

90. Can an interface extend a class?

No, a class can implement an interface but interface cannot extend a class. But an interface can extend one or more interfaces.

91. Can we declare an interface with the abstract keyword?

Yes, we can declare abstract keyword with interfaces but there is no need to write abstract keyword with interfaces.

92. What is default keyword in an interface?

By the help of default keyword, we can keep non-abstract methods in java interface. i.e with method body{}. This is a new feature of JDK 1.8.

Concrete methods can be declared as static also in JDK 1.8.

Conclusion is concrete methods must be either static or default.

Syntax of default keyword :

```
interface Test{  
    default void show(){  
        ----  
        ----  
    }  
}
```

93. What is marker interface?

Marker Interface in java is an interface with no fields or methods within it.

It is used to convey to the JVM that the class implementing this interface should be treated differently.

Eg: Serializable ,Cloneable etc..

94. Does interface extend Object class by default?

No, Interface does not extend Object class in java by default but all the classes extend Object class by default.

95. After compilation of interface program, .class file will be generated for every interface in java. true or false.?

True

96. What will happen if we do not initialize variables in an interface.?

Compile time error will occur because by default members will be treated as public static final, so it expects some value to be initialized.

97. What will happen if a class implements two interfaces and they both have a default method with same name and signature?

In this case, a conflict will arise because the compiler will not be able to link a method call due to ambiguity. You will get a compile time error.

98. What are the inheritances possible in java using extends and implements?

- ❖ A class can implement one or more interfaces.
- ❖ A class can extend only one class.
- ❖ An interface can extend one or more interfaces.

99. How can we access variables with same name defined in two interfaces from a class which implements both these interfaces?

By Using corresponding interface.variable_name we can access variables of corresponding interfaces.

100. Can we change the value of a field in interface after initialization?

No, Because all the fields of the interface are by default final.

101. What is the difference in Interface of Java7, Java8?

Interface in JAVA7

It accumulates only two things i.e.

- ❖ constant variables
- ❖ abstract methods

i.e.: We cannot provide implementation in interface

Interface in JAVA8

Now it has 4 things i.e

- ❖ constant variables
- ❖ abstract method
- ❖ default method
- ❖ static method

Method can have implementation in Interface from JAVA8 onwards by using default or static keyword.

102. Does Importing a package imports its sub-packages as well in Java?

In java, when a package is imported, its sub-packages aren't imported and developer needs to import them separately if required.

For example, if a developer imports a package `university.*`, all classes in the package named `university` are loaded but no classes from the sub-package are loaded. To load the classes from its sub-package (say `department`), developer has to import it explicitly as follows:

`Import university.department.*`

103. When the constructor of a class is invoked?

The constructor of a class is invoked every time an object is created.

104. Can a class have multiple constructors?

Yes, a class can have multiple constructors with different arguments. It is called constructor overloading. Based on the arguments passed while creating the object, the respective constructor will be invoked.

105. Can a digit form the first character of an identifier?

No. Only character, dollar symbol(\$) or underscore(_) is allowed

However after that any of these and/or digits can be used.

106. Why Strings in Java are called as Immutable?

In java, String objects are called immutable as once value has been assigned to a string, it can't be changed and if we change, a new object will be created in the string constant pool. The original string will not be changed.

107. What is encapsulation?

Encapsulation means wrapping up of states and behavior into a single unit. A fully encapsulated class must have the variables declared as private and must have public methods for accessing these variables(states). These are called getters and setters.

Example of fully Encapsulated class

```
public class Mobile {  
    private String model;  
    private int price;  
  
    public Mobile() {  
    }  
  
    public Mobile(String model, int price) {  
        this.model = model;  
        this.price = price;  
    }  
  
    public void setModel(String model) {  
        this.model = model;  
    }  
}
```

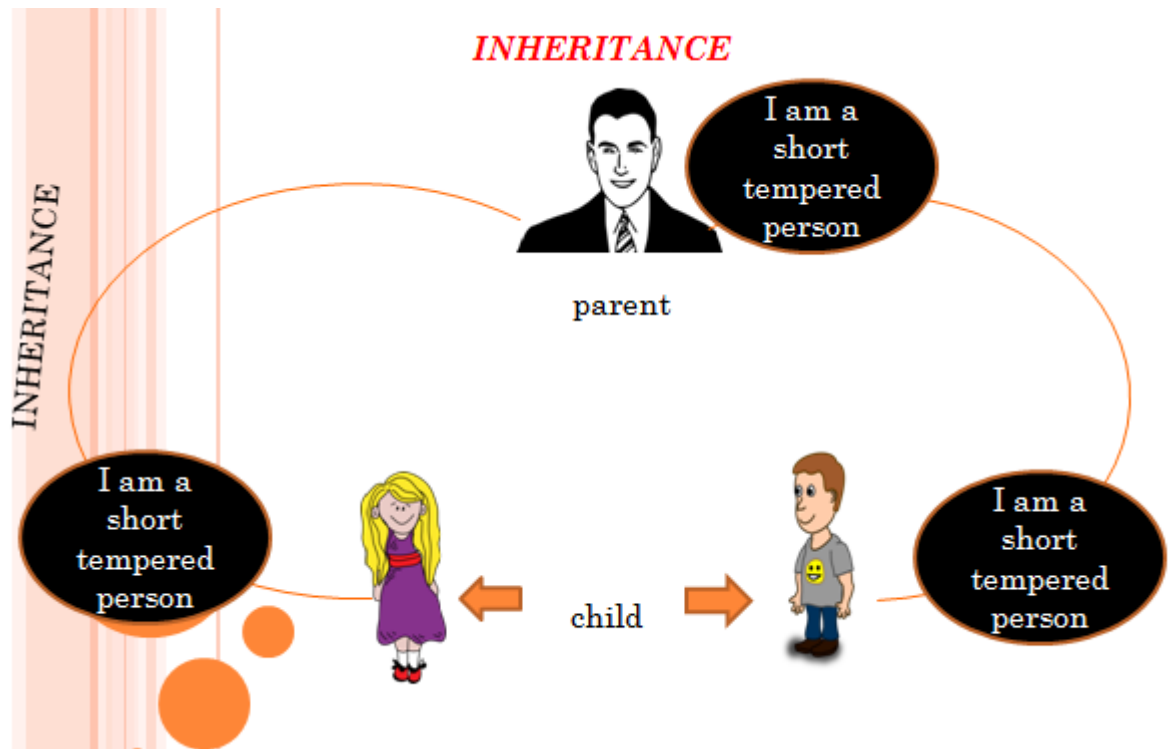
```
    public String getModel() {  
        return model;  
    }  
    public void setPrice(int price) {  
        this.price = price;  
    }  
  
    public int getPrice() {  
        return price;  
    }  
}
```

108. What is the advantage of fully encapsulated class?

We can give any validation logic inside the setter method. For eg: if the student age must be greater than 18 to join for a program, that logic can be placed inside the setter method.

109. What is Inheritance in Java?

Inclusion of states and behaviors of a base class in a sub class is called inheritance.



Eg:

```
class Bird {
    public void fly(){
        System.out.println("Birds flies using wings");
    }
}
class Parrot extends Bird{
}
class Test{
    public static void main(String args[]){
        Parrot p = new Parrot();
        p.fly();
    }
}
```

Birds flies using wings will be printed.

110. What do you achieve in inheritance?

Code reuse and dynamic polymorphism

111. How to use Inheritance in Java?

You can use Inheritance in Java by extending a class and by implementing interfaces. Java provides two keywords extends and implements to achieve inheritance.

So there are two types of inheritance in Java. Implementation Inheritance and Interface Inheritance. A class extending another class is called Implementation Inheritance and a class implementing one or more interfaces is called Interface Inheritance.

Don't use inheritance just to get code reuse. Derive a class from another class only if there is an 'IS A' relationship between these two classes in such a way that child class 'IS A' parent class.

Eg: Dog and Animal

Rose and Flower

Car and Vehicle

Overuse of implementation inheritance (uses the "extends" key word) can break all the subclasses, if the super class is modified.

112. What is abstraction?

Hiding internal details(implementations) and exposing only the essential details is known as abstraction.

For eg: A mobile phone. To make a call we have a dialer to enter the number but when we talk we do not know exactly how the call is getting connected.

In java , we use abstract class and interface to achieve abstraction.

113. What is polymorphism?

One having many forms is the simple definition for polymorphism.



- In Shopping mall behave like customer
- In Bus behave like Passenger
- In School behave like Student
- At home behave like son

In java there are two types of polymorphism

- ❖ Static polymorphism or method overloading
- ❖ Dynamic polymorphism

114. What is method overloading?

- ❖ Same operation in different ways for different data.

```
public void eat(){
```



```
}
```

```
public void eat(Spoon s){
```



- ❖ A class having more than one method with same name is called method overloading.

- ❖ The rule for method overloading is either the number of arguments or order of arguments or type of arguments must be different.
- ❖ Method overloading increases the readability of the program.

Eg:

```
public class AreaCalculator {
    public int getArea(int l,int b){
        return l*b;
    }
    public double getArea(int r){
        return 3.14*r*r;
    }
}
class TestArea{

    public static void main(String[] args) {
        AreaCalculator ar = new AreaCalculator();
        int recArea = ar.getArea(5, 10);
        double cirArea = ar.getArea(3);
        System.out.println("the area of rectangle:
"+recArea);
        System.out.println("the area of circle:
"+cirArea);
    }
}
```

115. What is dynamic polymorphism?

It is the ability of an object variable of one type to refer different types.

Suppose we have a class called Animal and two derived classes like Dog and Cat.

Now we can create an object of Dog like this and later we can create an object of Cat with the same reference variable.

`Animal a = new Dog();`



a is the variable of type Animal, it is referring Dog.

`a = new Cat();`



a is reused now. Now it is refering Cat.

Note: Dog & Cat must be the subclass of Animal.

116. Can you extend String class?

No, we can't extend String class because String is a final class.

117. Which kind of memory is used for storing instance variables and local variables?

Local variables are stored in stack whereas instance variables are stored in heap.

118. Is null a keyword?

The null is not a keyword. null is a literal like true and false.

119. Who adds the default constructor if you don't provide one?

Compiler.

120. Can you instantiate abstract class?

No.

121. What is Anonymous Inner Class in Java?

It is an inner class without a class name and for which only a single object is created. An anonymous inner class can be useful when making an instance of an object and same time overriding a method of a class or interface.

```
interface Calculator {
    public void calculate();
}

public class CalculatorDemo {

    public static void main(String[] args) {
        Calculator calculator = new Calculator() {
            @Override
            public void calculate() {
                System.out.println("Do the
calculation");
            }
        }

        calculator.calculate();
    }
}
```

122. Can we call the garbage collector explicitly ?

We can call `System.gc()` but it doesn't guarantee that the Garbage Collector will clean the memory.

123. Can we overload private method in Java?

Yes, you can overload a private method in Java. Only thing is either number of arguments or type of arguments or order of arguments must be different.

124. Can we override a private method in Java?

No, you cannot override a private method in Java because the private method is not inherited by the subclass. In fact, visibility of a private method is only within the class.

125. What is the difference between constructor and regular methods ?

- ❖ Constructor gets called only once per object creation but regular method can be called any number of times.
- ❖ Constructors must have the same name as the class name but regular method can have any name.
- ❖ Constructor doesn't have a return type. Regular method must have a returns type. If the method doesn't return anything, then void should be given.

126. What is toString() method?

Every class in java is child of Object class either directly or indirectly.

Object class contains toString() method. We can use toString() method to get string representation of an object. Whenever we try to print the Object reference then internally toString() method is invoked. So memory address of the object will be printed. But If you want to print the meaningful data, you have to override the toString() method in your class.

Syntax of Object class toString() method:

```
public String toString() ;
```

127. Name few Java IDE's?

Eclipse and NetBeans, IntelliJ IDEA etc.

128. What is the main difference between pass-by-reference and pass-by-value?

In pass-by-reference the caller and the callee use the same variable for the parameter. A change in callee's parameter variable, reflects in caller's variable too.

In pass-by-value the caller and callee have two independent variables with the same value. A change in callee's parameter variable, will not get reflected in caller's variable.

In Java no matter what type of argument you pass the corresponding parameter (primitive variable or object reference) will get a copy of that data, which is exactly how pass-by-value (or copy-by-value) works.

Primitive variables :

```
public void methodOne() {  
    int a = 10;  
    int b = methodTwo(a);  
    // At this point value of a is still 10  
    // value of b is 11  
}
```

```

public int methodTwo(int c) {
    c++;
    return c; // modifies the copy c but not the
original.
}

```

Object references :

```

public void methodOne(){
    Car c = new Car("black")
    //At this point color is black
    methodTwo(c);
    //At this point color is white
}

public void methodTwo(Car d) {
    d.setColor("white");
    // color is white
    // modifies the original object through copied
reference
}

```

129. Can we have multiple public classes in a java source file.?

No

130. What are the non-access modifiers you know?

- ❖ abstract
- ❖ static
- ❖ final
- ❖ synchronized
- ❖ transient

131. What is the use of the final keyword in Java?

The final keyword can be used with a class, method, and variables. If it is used with class then it prevents inheritance by not allowing you to create subclasses.

If it is used with methods then it prevents overriding, you cannot override a final method in Java.

If it is used with variables then they are treated as constant because you cannot change their value once assigned.

132. Can we make a variable final in Java? What is different between a normal variable and final variable?

Yes, you can make a variable final in Java. The difference between normal variable and final variable comes from multiple assignments. You can re-assign value to a normal variable but you cannot change the value of a final variable once assigned.

133. Can we make a method final in Java? Explain the difference between normal method and final method?

Yes, you can make a method final in Java. The difference is in the fact that you can override a non-final method, provided it's not private or static but you cannot override a final method in Java.

134. What is the main difference between abstract methods and final methods?

Abstract methods must be overridden in the sub classes and final methods are not at all eligible for overriding.

135. Where can we initialize a final static variable if it is not initialized at the time of declaration?

In any one of static initialization blocks.

136. Can we make local variable final in Java?

Yes, you can make local variable final in Java.

137. Can we make an array final in Java? Can you change its elements?

Yes, you can make an array final in Java and you can change its elements as well.

138. Can you make a Collection final in Java e.g. ArrayList? What is the impact?

Yes, you can make a Collection final in Java. The impact is nothing but the final variable cannot be swapped with another Collection, but you can still add, remove and update elements in ArrayList or any collection classes.

139. Can we make a static method final in Java?

Yes, you can make a static method final in Java.

140. Is it possible abstract final for a method?

No. If you are declaring a method as final, you can't override that method.

If you have abstract method, you must override that method. So abstract and final are illegal combination. When you give abstract final for a method compiler will show an error.

141. Is it possible to have an abstract method in a final class?

No, it's not possible to have an abstract method in a final class in Java because as soon as you declare an abstract method in a Java class, the class automatically becomes an abstract class and you cannot make an abstract class final in Java. It is an illegal combination and you will get a compile time error.

142. What is the point of “final class” in Java?

One scenario where final is important, when you want to prevent inheritance of a class, for security reasons. This allows you to make sure that code you are running cannot be overridden by someone.

The best example is

```
public final class String
```

which is an immutable class and cannot be extended. Of course, there is more than just making the class final to be immutable.

143. Difference between Heap and Stack Memory in Java JVM?

Heap memory is shared by all threads of Java application but Stack memory is local to each thread. Objects are created in heap memory but method frames are stored in Stack memory, and size of heap space is much bigger than the small size of Stack in Java.

144. Difference between StringBuffer, StringBuilder and String in Java?

- ❖ The String class is an immutable class whereas StringBuffer and StringBuilder classes are mutable. The StringBuilder class is introduced since JDK 1.5.
- ❖ StringBuffer is thread safe because all methods of StringBuffer are synchronized, StringBuilder is non-synchronized i.e. not thread safe.
- ❖ StringBuffer is less efficient than StringBuilder. i.e. StringBuilder is faster than StringBuffer.

145. When to use the String, StringBuffer, and StringBuilder in Java

- ❖ Use String if you need text data which is relatively constant e.g. names, config parameters etc.

- ❖ Use StringBuilder if are doing lots of String concatenation e.g. you are generating dynamic String by using programming logic.
- ❖ Use StringBuffer when your String manipulation code is likely to be executed by multiple threads and you are sharing the instance of same StringBuffer.

146. Difference between a class and an interface in Java?

- ❖ A class can be instantiated by creating its objects. An interface is never instantiated.
- ❖ The members of a class can have the access specifiers like public, private, protected. But the members of an interface are always public.
- ❖ A class can implement any number of interfaces but can extend only one class. An interface can extend any number of interfaces.
- ❖ A class has constructors defined inside it to get the variable initialized. But, an interface does not have any constructors as there are no fields to be initialized.

147. Differences Between while and do-while Loop?

- ❖ The while loop checks the condition at the starting of the loop and if the condition is satisfied statement inside the loop, is executed. In do-while loop, the condition is checked after the execution of all statements in the body of the loop.
- ❖ If the condition in a while loop is false not a single statement inside the loop is executed, and if the condition in 'do-while' loop is false then also the body of the loop is executed at least once before the condition is tested.

148. How many ways to Compare Two Strings in Java ?

- ❖ String comparison using equals() method
- ❖ String comparison using equalsIgnoreCase method
- ❖ String comparison using compareTo method
- ❖ String comparison using ==

149. Difference between equals and equalsIgnoreCase?

- ❖ equals() method compare two Strings for content equality. So if two string contains same letters, in same order and in same case they will be equal by equals() method. equals() method is defined in Object class and String class overrides that for character based comparison. For example for two Strings "Java" and "Java", equals() will return true but for "Java" and "JAVA" it will return false.
- ❖ equalsIgnoreCase() is more liberal than equals and compare two strings ignoring their case. So if two String contains same characters and in same order despite of their case e.g. lower case, upper case they will be equal by equalsIgnoreCase(). For example "Java" and "JAVA" two Strings will be same by equalsIgnoreCase() and it will return true.

150. What is difference between an int and Integer in Java?

- ❖ int, being a primitive data type has got less flexibility. We can only store the binary value of an integer in it.
- ❖ Since Integer is a wrapper class for int data type, it gives us more flexibility in storing, converting and manipulating an int data.
- ❖ Integer is a class and thus it can call various in-built methods defined in the class.

151. Why String is not a primitive data type in Java?

String is not a primitive data type. String is group of characters put together in other words its array of characters.

String in Java is itself is a class and has its own methods to manipulate and operate over object of String class.

Primitive data types has limitation that they have fixed size and can hold data of that type only but String can vary in size and it can hold any type of data using its wrapper classes and it is one of reason why STRING IS NON-PRIMITIVE data type.

152. What is the difference between primitive type and reference type in Java?

In Java there are 8 **primitive types** which are basic data types :

- ❖ byte
- ❖ short
- ❖ int
- ❖ long
- ❖ float
- ❖ double
- ❖ char
- ❖ boolean

reference types are any instantiable class as well as arrays.

Class types, Object types and Reference types all mean exact same; an object of a class. Examples : Object of a class Car, String, Integer,etc.

- ❖ Every variable irrespective of whether it is primitive or reference type is stored in the computer memory.
- ❖ For a primitive type of variable if you visit it's memory location you will find the value of the variable.

```
int x = 3;
```

Now if you visit the location where x is stored you can find the value 3.

For a reference type of variable if you visit it's memory location unlike the primitive type you will find a memory address pointing to other location and not the values of variables in Object. This memory address points to a location where the details of Object and values of variables are stored.

153. Can static method access instance variables ?

Though Static methods cannot access the instance variables directly, they can access them using object.

154. What will be in the first line of any constructor?

The first line of any constructor will have either this() or super() keyword.

155. Can abstract class have static variable in it ?

Yes, an abstract class can have static variables in it.

156. How an object eligible for garbage collection in Java?

An object is eligible to be garbage collected if it's reference is lost from the program during execution. i.e Un-referenced objects are suitable candidates for garbage collection.

157. What is static and dynamic class loading in Java?

❖ **Static Class Loading:**

Classes are statically loaded with Java's "new" operator.

```
class MyClass {  
    public static void main(String args[]) {  
        Car c = new Car();  
    }  
}
```

A **NoClassDefFoundException** is thrown if a class is referenced with Java's "new" operator (i.e. static loading) but the runtime system cannot find the referenced class.

❖ **Dynamic Class Loading:** Loading classes use `Class.forName ()` method.

Dynamic class loading is done when the name of the class is not known at compile time.

158. How java becomes robust?

- ❖ Java provides multi-platformed environment.
- ❖ Java provides high reliability in the design.
- ❖ Java is a strictly typed language.
- ❖ Java checks the code at runtime.
- ❖ Good Memory Management.

159. What is the Difference Between JAR and WAR Files?

The main difference between JAR and WAR Files is their content. The JAR files are the files that have Java class files, associated metadata and resources aggregated into a single file to execute a Java application.

Whereas, the WAR files are the files that contain Servlet, JSP, HTML, JavaScript and other files necessary for developing web applications.

160. What is ArrayStoreException in java? When you will get this exception?

`ArrayStoreException` is a run time exception which occurs when you try to store non-compatible element in an array object. The type of the elements must be compatible with the type of array object. For example, you can store only string elements in an array of strings. if you try to insert integer element in an array of strings, you will get `ArrayStoreException` at run time.

Eg:

```
public class MainClass {
    public static void main(String[] args)
    {
        Object[] stringArray = new String[5];
        //No compile time error : String[] is auto-upcasted
        to Object[]

        stringArray[1] = "JAVA";

        stringArray[2] = 100;    //No compile time
        error, but this statement will throw
        java.lang.ArrayStoreException at run time

        //because we are inserting integer element
        into an array of strings
    }
}
```

161. Can you pass the negative number as an array size?

No. You can't pass the negative integer as an array size. If you pass, there will be no compile time error but you will get `NegativeArraySizeException` at run time.

```
public class MainClass
{
    public static void main(String[] args)
    {
        int[] array = new int[-5];    //No
        compile time error

        //but you will get
        java.lang.NegativeArraySizeException at run time
    }
}
```

162. Can you change the size of the array once after creating it?

No. You can't change the size of the array once after creating the array.

163. What is an anonymous array? Give example?

Anonymous array is an array without reference.

```
public class MainClass
{
    public static void main(String[] args)
    {
        //Creating anonymous arrays
        System.out.println(new int[]{1, 2, 3, 4, 5}.length);
        //Output : 5
        System.out.println(new int[]{21, 29, 65, 24, 21}[1]);
        //Output : 29
    }
}
```

164. What is the difference between int[] one and int one[] ?

Both are the legal ways to declare the arrays in java.

But here:

int[] one, two; // Both one and two are arrays of type int

int one[], two; // one is an array, but two is just an int variable

165. There are two array objects of int type. One is containing 100 elements and another one is containing 10 elements. Can you assign array of 100 elements to an array of 10 elements?

Yes, you can assign array of 100 elements to an array of 10 elements provided they should be of same type. While assigning, compiler checks only type of the array not the size.

```
public class MainClass
{
    public static void main(String[] args)
```

```

    {
        int[] a = new int[10];

        int[] b = new int[100];

        a = b;    //Compiler checks only type,
not the size
    }
}

```

166. What are the different ways of copying an array into another array?

There are four methods available in java to copy an array.

- 1) Using for loop
- 2) Using Arrays.copyOf() method
- 3) Using System.arraycopy() method
- 4) Using clone() method

Copying An Array Using for Loop :

```

public class ArraysInJava {
    public static void main(String[] args)
    {
        int[] a = {12, 21, 0, 5, 7};
        int[] b = new int[a.length];
        for (int i = 0; i < a.length; i++)
        {
            b[i] = a[i];
        }
        for(int i=0;i<b.length;i++){
            System.out.println(b[i]);
        }
    }
}

```


Copying An Array Using copyOf() Method of java.util.Array Class :

```
public class ArraysInJava {
    public static void main(String[] args)
    {
        int[] a = {12, 21, 0, 5, 7};
        int[] b = Arrays.copyOf(a, a.length);
        //Printing elements of array 'b'
        for (int i = 0; i < b.length; i++)
        {
            System.out.println(b[i]);
        }
    }
}
```

Copying An Array Using clone() Method :

All arrays will have clone() method inherited from java.lang.Object class. Using this method, you can copy an array.

```
public class ArraysInJava {
    public static void main(String[] args)
    {
        int[] a = {12, 21, 0, 5, 7};

        int[] b = a.clone();

        //Printing elements of array 'b'

        for (int i = 0; i < b.length; i++)
        {
            System.out.println(b[i]);
        }
    }
}
```

Copying An Array Using arraycopy() Method Of System Class :

```
public class ArraysInJava
{
    public static void main(String[] args)
    {
        int[] a = {12, 21, 0, 5, 7};

        int[] b = new int[a.length];

        System.arraycopy(a, 0, b, 0,
a.length);

        //Printing elements of array 'b'

        for (int i = 0; i < b.length; i++)
        {
            System.out.println(b[i]);
        }
    }
}
```

167. How do you check the equality of two arrays in java? OR How do you compare the two arrays in java?

- 1) Iterative Method :
- 2) Using Arrays.equals() Method :

Iterative Method :

```
public class EqualityOfTwoArrays
{
    public static void main(String[] args)
    {
        int[] arrayOne = {2, 5, 1, 7, 4};
        int[] arrayTwo = {2, 5, 1, 7, 4};
    }
}
```

```

        boolean equalOrNot = true;

        if(arrayOne.length==arrayTwo.length)
        {
            for (int i=0;i<arrayOne.length;i++)
            {
                if(arrayOne[i] != arrayTwo[i])
                {
                    equalOrNot = false;
                }
            }
        }
        else
        {
            equalOrNot = false;
        }

        if (equalOrNot)
        {
            System.out.println("Two Arrays Are
Equal");
        }
        else
        {
            System.out.println("Two Arrays Are Not
equal");
        }
    }
}

```

Using Arrays.equals() Method :

```

public class EqualityOfTwoArrays
{
    public static void main(String[] args)
    {
        String[] s1 = {"java", "j2ee", "jms",
"ORM"};
    }
}

```

```

        String[] s2 = {"jsp", "spring", "jdbc",
"ORM"};

        String[] s3 = {"java", "j2ee", " jms ", "
ORM "};

        System.out.println(Arrays.equals(s1, s2));
//Output : false
        System.out.println(Arrays.equals(s1, s3));
//Output : true
    }
}

```

168. How do you sort the array elements?

You can sort the array elements using Arrays.sort() method. This method internally uses quick sort algorithm to sort the array elements.

```

public class MainClass
{
    public static void main(String[] args)
    {
        int[] a = new int[]{45, 12, 78, 34, 89, 21};
        Arrays.sort(a);
        System.out.println(Arrays.toString(a));
        //Output : [12, 21, 34, 45, 78, 89]
    }
}

```

169. What value does array elements get, if they are not initialized?

They get default values.

170. What are the different ways to iterate over an array in java?

1) Using normal for loop

```
public class MainClass
{
    public static void main(String[] args)
    {
        int[] a = {45, 12, 78, 34, 89, 21};
        for (int i = 0; i < a.length; i++)
        {
            System.out.println(a[i]);
        }
    }
}
```

2) Using extended for loop

```
public class MainClass
{
    public static void main(String[] args)
    {
        int[] a = {45, 12, 78, 34, 89, 21};
        for (int i : a)
        {
            System.out.println(i);
        }
    }
}
```

Also other loops like while and do while can be used to iterate over an array.

171. What are the drawbacks of the arrays in java?

The main drawback of the arrays is that arrays are of fixed size. You can't change the size of the array once you create it. Therefore, you must know how many elements you want in an array before creating it. Only you can change the value of the elements.

172. How to convert string containing an int value to int?

```
int n = Integer.parseInt("10");
```

173. How to split a string with white space characters?

We can simple do split using regular expression. "\\s" stands for white space characters.

```
String s="I love jThread";  
String strArray[]=s.split("\\s");
```

174. What is subString() in String class?

The Java String.substring() method returns a new string that is a substring of this string. substring() method has overloaded methods and comes in two variants:

```
String substring(int beginIndex)  
String substring(int beginIndex, int endIndex)
```

Where method arguments are:

beginIndex – the beginning index, inclusive.

endIndex – the ending index, exclusive.

Eg:

```
String s="jThread Trivandrum";  
System.out.println(s.substring(8));//Trivandrum  
System.out.println(s.substring(0, 7));//jThread
```

How many String objects got created in below code?

1. `String s1 = new String("Hello");`
2. `String s2 = new String("Hello");`

The answer is 3.

First – line 1, “Hello” object in the string pool.

Second – line 1, new String with value “Hello” in the heap memory.

Third – line 2, new String with value “Hello” in the heap memory.

Here “Hello” string from string pool is reused.

175. How to convert String to byte array?

We can use String `getBytes()` method to convert String to byte array.

Eg:

```
String str = "jThread";  
byte[] byteArr = str.getBytes();
```

176. What is `replaceAll()` method in String?

The Java String `replaceAll()` method returns a string after it replaces each substring of that matches the given regular expression with the given replacement.

Use `String.replaceAll(String regex, String replacement)` to replace all occurrences of a substring (matching argument `regex`) with replacement string.

❖ Replace all occurrences of a substring or word

Eg:

```
String s="jThread Trivandrum";  
String s1=s.replaceAll("Trivandrum", "Bengaluru");  
System.out.println(s1);  
//jThread Bengaluru
```

❖ Replace all white spaces

Eg:

```
String s="jThread Trivandrum";  
String s1=s.replaceAll("\\s", "");  
  
System.out.println(s1);//jThreadTrivandrum
```

177. What is the difference between String literal and String object ?

```
String str1 = new String("hello world");
```

```
String str2 = "hello world";
```

In above example, both are used to create strings, but later is recommended which uses string literals. String literals always go to string pool.

When we create string with new keyword, two objects will be created i.e. one in the Heap Area and another in the String constant pool. The created string object reference always points to heap area object.

To get the reference of same object created in string pool, use intern() method.

178. What is java String intern() method?

The String.intern() returns a reference to equal string literal present in string pool.

As we know that all string literals are automatically created in String pool, so intern() method is applicable to String objects created via 'new' keyword.

String intern() is native method. By the help of intern() method, we can get the reference of corresponding String constant pool object of an original string object.

Eg:

```
public class StringInternMethodExample
{
    public static void main(String[] args) {
        //String object in heap
        String str1 = new String("hello world");

        //String literal in pool
        String str2 = "hello world";

        //String literal in pool
        String str3 = "hello world";

        //String object interned to literal
        //It will refer to existing string literal
        String str4 = str1.intern();
        System.out.println(str1 == str2);           //false
        System.out.println(str2 == str3);
        //true
        System.out.println(str2 == str4);
        //true
    }
}
```

179. How can we make String upper case or lower case?

We can use String class's toUpperCase() and toLowerCase() methods to get the String in all upper case or lower case.

```
Eg:  String s="hello";

      String s1="HI";
      System.out.println(s.toUpperCase()); //HELLO
      System.out.println(s1.toLowerCase()); //hi
```

180. Why Char array is preferred over String for storing password?

String is immutable in Java and stored in String pool. Once it's created it stays in the pool until unless garbage collected, so even though we are done with password it is available in memory for longer duration and there is no way to avoid it. It's a security risk because anyone having access to memory dump can find the password as clear text.

If we use a char array to store password, we can set it to blank once we are done with it. So we can control on how long it's available in memory that avoids the security threat with String.

181. Is String Thread Safe?

Yes Strings are thread safe. Since String is immutable it can safely shared between many threads. We don't need to synchronize String operation externally.

182. What is the output of below code ?

```
String s1 = new String("abc");  
String s2 = new String("abc");  
System.out.println(s1 == s2);
```

It will print false because we are using new operator to create String, so it will be created in the heap memory and both s1, s2 will have different reference. If we create them using double quotes, then they will be part of string pool and it will print true.

183. What is the output of below code?

```
String s1 = "abc";  
String s2 = new String("abc");  
s2.intern();  
System.out.println(s1 ==s2)
```

The output will be false. We know that intern() method will return the String object reference from the string pool, but since we didn't assign it back to s2, there is no change in s2 and hence both s1 and s2 are having different reference. If we change the code to s2 = s2.intern(); then output will be true.

184. Can you create an object without using new operator in Java?

Yes, We can create an object without using new operator. There are some other ways to create objects than using new operator.

a) Using newInstance() method of Class

```
Class c = Class.forName("packageName.MyClass");  
MyClass object = (MyClass) c.newInstance();
```

b) Using clone() method.

```
MyClass object1 = new MyClass();  
MyClass object2 = object1.clone();
```

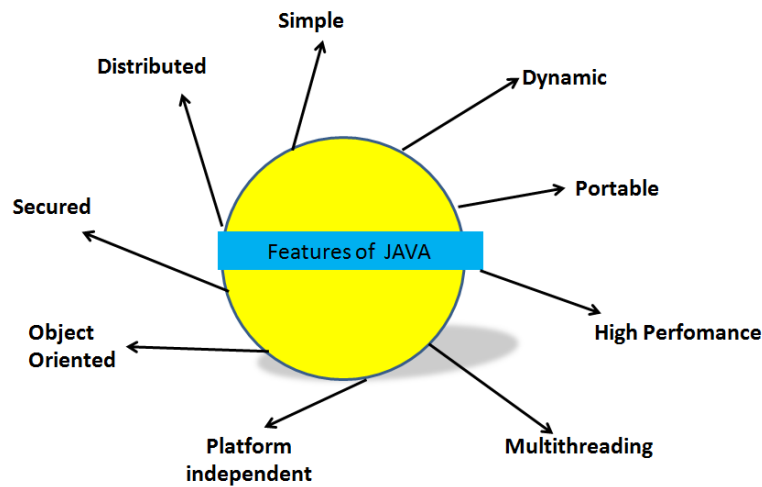
c) Using object deserialization

```
ObjectInputStream inStream = new  
ObjectInputStream(anInputStream );  
MyClass object = (MyClass) inStream.readObject();
```

d) Creating string and array objects :

```
String s = "string object";  
int[] a = {1, 2, 3, 4};
```

185. What are the features of Java?



186. What is the benefit of object oriented programming language?

- ❖ It provides a clear modular structure for programs which makes it good for defining abstract data types in which implementation details are hidden.
- ❖ Objects can also be reused within an across applications. The reuse of software also lowers the cost of development. More effort is put into the object-oriented analysis and design, which lowers the overall cost of development.
- ❖ It makes software easier to maintain. Since the design is modular, part of the system can be updated in case of issues without a need to make large-scale changes.
- ❖ Reuse also enables faster development. Object-oriented programming languages come with rich libraries of objects, and code developed during projects is also reusable in future projects.

- ❖ It provides a good framework for code libraries where the supplied software components can be easily adapted and modified by the programmer. This is particularly useful for developing graphical user interfaces.
- ❖ Better Productivity as OOP techniques enforce rules on a programmer that, in the long run, help her get more work done; finished programs work better, have more features and are easier to read and maintain. OOP programmers take new and existing software objects and "stitch" them together to make new programs. Because object libraries contain many useful functions, software developers don't have to reinvent the wheel as often; more of their time goes into making the new program.

187. How is Java more secure than other languages?

❖ Own Memory Management

The memory management mechanism is unique and is owned by Java. There is no need for manual intervention for garbage collection and everything is handled automatically. There is no need for a headache to free the memories. It drastically reduces the programmer overhead. Therefore the programmer's hand must be free from memory management. Relieving the memory in Java is the job of JVM.

❖ No Pointers

There is no support for the pointers concept in Java. Some of the arbitrary memory locations can be addressed with the help of

pointers for doing read and write operations which are unauthorized. This does not serve the purpose of being secured. That is why users do not use the concept of pointers.

❖ **Access Specific Keywords**

There is another benefit for Java security and it is nothing but having an access specific keyword. If access to a method is what programmer wants to give to other functions then the public keyword must be used. If he/she wants to hide the information then private keywords must be used. Access security issues can be controlled by the programmer like the above-mentioned line. For avoiding data to be overridden the programs can use the final keyword.

❖ **Compile Time Checking**

It is more secure because of the ability of compile-time checking. For instance, if a method which is unauthorized and wanting to access a variable which private then the JVM will catch the error during compile time. For avoiding system crash JVM catches lots of errors which is actually a great thing. Two different results are produced with the help of two different Java compilers. Memory locations cannot be accessed other than the array as it checks array bounds carefully.

❖ **Exception Handling**

This concept present in Java actually means that it can catch the results which are unexpected through exception handling and report the error to the programmer. Until the rectification of the error by the programmer this concept will not allow us to run the code. Thereby proving Java's security.

188. What is a best real life example of Java?

- ❖ Android apps
- ❖ Server Apps for different industries
- ❖ Scientific Applications
- ❖ J2ME Apps
- ❖ Java Web applications
- ❖ Enterprise Applications
- ❖ Desktop GUI Applications
- ❖ Big Data technologies

189. What are the different platforms of java?

- ❖ Java Platform, Standard Edition (Java SE)
- ❖ Java Platform, Enterprise Edition (Java EE)
- ❖ Java Platform, Micro Edition (Java ME)
- ❖ Java FX

190. What is Java SE?

Java Standard Edition(or Core java) to develop desktop based applications such as Swing/AWT applications or any console based applications. The app must be installed on client's desktop.

191. Which company invented Java?

Java was originally developed by James Gosling at Sun Microsystems (which has since been acquired by Oracle) and released in 1995 as a core component of Sun Microsystems' Java platform.

192. What are the principles of oops?

- ❖ Encapsulation
- ❖ Abstraction
- ❖ Inheritance
- ❖ Polymorphism

193. What happens if the parent and the child class have a field with same identifier?

Super class field member will be hidden in the sub class and super class field could be accessed using super keyword.

194. Can a super class access sub class members?

No. Parent class cannot access the child class members. Super class reference variable cannot see subclass object members.

195. What is Write Once, Run Anywhere (WORA)?

WORA refers to the ability of a computer program to run across any operation system or platform.

196. Is Java a pure/fully object oriented language?

Java has primitive data types which are not objects. However Java could be considered as fully object oriented language as it has wrapper classes for all the primitive data types.

197. What is classloader?

The classloader, a subsystem of JVM, is used to load classes and interfaces.

Types of classloader.

There are many types of classloaders e.g.

- ❖ Bootstrap,
- ❖ Extension,
- ❖ System,
- ❖ Plugin.

198. Explain memory leak in Java.?

Memory leak in Java refers to a situation where some objects are not used by the application any more, but GC fails to recognize them as unused. As a result, these objects remain in memory indefinitely, reducing the amount of memory available to the application.

199. Does Java garbage collector cleans both heap and stack memory?

GC sweeps heap memory only. Usually, stack memory is collected automatically when the execution path reaches the end of the scope.

200. Can a Java source file have more than one class declaration?

Yes. However only one class can be declared as public and the java file name should be exactly same as the public class name along with .java extension.

201. Can we have a class name same as an interface name under the same package?

No.

202. Difference Between ClassNotFoundException & NoClassDefFoundError.?

Both of these related to missing classes in the classpath, however the main difference lies how it originate.

ClassNotFoundException occurs when you try to load a class at runtime by using `Class.forName()` or `loadClass()` and requested class is not present in classpath. It is thrown when an application tries to load in a class through its name, but no definition for the class with the specified name could be found.

NoClassDefFoundError is encountered when the class was available at compile time but not during runtime.

203. What are the Object class methods?

- ❖ protected `Object clone()` throws `CloneNotSupportedException`

Creates and returns a copy of this object.

- ❖ public boolean `equals(Object obj)`

Indicates whether some other object is "equal to" this one.

- ❖ protected void `finalize()` throws `Throwable`

Called by the garbage collector on an object when garbage collection determines that there are no more references to the object.

❖ `public final Class getClass()`

Returns the runtime class of an object.

❖ `public int hashCode()`

Returns a hash code value for the object.

❖ `public String toString()`

Returns a string representation of the object.

❖ The `wait()`, `notify()` and `notifyAll()` methods of `Object` all play a part in synchronizing the activities of independently running threads in a program.

204. What is enum in java?

It is a special type used to define collection of constants.

- ❖ Having one of only a few pre-defined values for a variable.
- ❖ One value from an enumerated list.
- ❖ Reduces the bug in the code.

For example your car application you can restrict the wheel selection to `FOURWHEEL`, `SIXWHEEL`.

Compiler stops from a selection of any other types.

```
enum CarWheel { FOURWHEEL , SIXWHEEL };
```

205. Name the interfaces that Java String class implements.?

String implements `Serializable`, `Comparable` and `CharSequence` interfaces.

206. what happens if beginIndex is equal to length of the String when calling substring(int beginIndex)?

substring() method returns an empty string and does not throw StringIndexOutOfBoundsException.

207. When substring() method throws StringIndexOutOfBoundsException?

Substring() method will throw StringIndexOutOfBoundsException in following cases:

When beginIndex or endIndex is negative

When beginIndex is larger than endIndex

When beginIndex or endIndex larger than length of String.

208. Where String pool is located?

Before Java 7, the JVM placed the Java String Pool in the PermGen space, which has a fixed size — it cannot be expanded at runtime and is not eligible for garbage collection.

From Java 7 onwards, the Java String Pool is stored in the heap space, which is garbage collected by the JVM. The advantage of this approach is the reduced risk of OutOfMemory error because unreferenced Strings will be removed from the pool, thereby releasing memory.

209. What is Object.finalize() method?

finalize() is a method in Object class. It's signature is protected void finalize(). This is the official statement: "Called by the garbage collector on an object when garbage collection determines that there are no more references to the object".

EXCEPTION HANDLING

1. What is Exception in Java?

Deviation from the normal flow of the program is called exception.

Exception happens during run time.

2. What are the two types of Exceptions in Java? What are the differences between them?

Java has two types of exceptions: checked exceptions and unchecked exceptions.

Checked Exception:



- ❖ Checked exceptions are exceptions that are warned by the compiler.

That means the compiler checks them during compilation to see whether the programmer has handled that exception or not.

- ❖ If these exceptions are not handled, you will get compilation error.

- ❖ So we have to handle that exception by using try-catch block or the programmer should declare the exception using throws keyword.

Eg:FileNotFoundException,IOException,SQLException etc.

Unchecked Exception:



- ❖ Unchecked exceptions are exceptions that are not warned by the compiler.
- ❖ Unchecked exceptions are not checked at compile time.

3. What is the difference between Exception and Error in java ?

Exception and Error classes are subclasses of Throwable. An Exception is something which can be handled. When an exception happens during run time there are ways to recover from that. But an Error is a serious situation which one should not try to handle. Because we cannot recover from this using any handling techniques.

4. Can we write only try block without catch and finally blocks?

No, it will result in compile time error. The try block must be followed by either catch or finally block. You can remove either catch block or finally block but not both.

5. There are three statements in a try block – statement1, statement2 and statement3. After that there is a catch block to catch the exception which might be thrown from the try block. Assume that exception has occurred in statement2. Does statement3 get executed or not?

No. Once a try block throws an exception, remaining statements will not get executed. The control goes directly to the catch block.

6. What is unreachable catch block error?

When you have multiple catch blocks against a try, the order of catch blocks must be from most specific to general ones. i.e sub class exceptions must come first and then super classes later. If you violate this rule you will get unreachable catch block error. i.e If you keep super class exceptions first and then sub class exceptions later unreachable catch block error occurs.

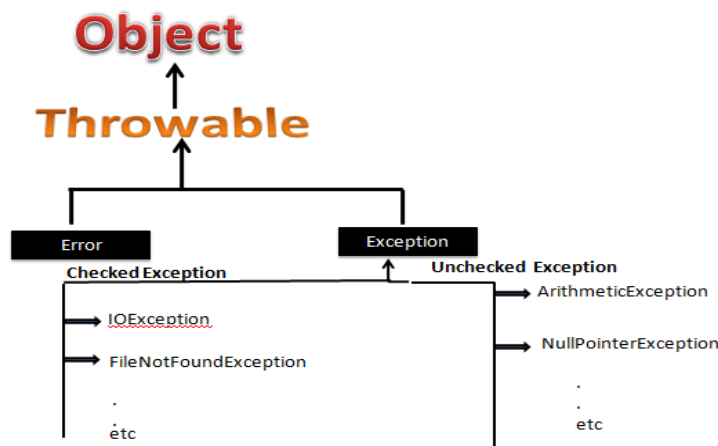
7. Does finally block get executed if either try or catch blocks are returning the control?

Yes, finally block will always be executed no matter whether try or catch blocks are returning the control or not.

8. Why it is always recommended that clean up operations like closing the DB resources, closing streams etc. to keep inside a finally block?

Because finally block is always executed whether exceptions are raised in the try block or not and raised exceptions are caught in the catch block or not. By keeping the cleanup operations in finally block, you will ensure that those operations will always be executed irrespective of whether exception occurs or not.

9. Explain Java Exception Hierarchy?



10. How can you handle an exception?

By using a try-catch-finally statement:

```
try {  
    // ...  
} catch (ExceptionType1 ex) {  
    // ...  
} catch (ExceptionType2 ex) {  
    // ...  
} finally {  
    // ...  
}
```

- ❖ The block of code in which an exception may occur is enclosed in a try block. This block is also called “protected” or “guarded” code.
- ❖ If an exception occurs, the catch block that matches the exception being thrown is executed, if not, all catch blocks are ignored.
- ❖ The finally block is always executed, irrespective of whether an exception was thrown or not inside the try block; finally block is optional.

11. What will happen to the Exception object after exception handling ?

The Exception object will be garbage collected later.

12. What are the rules of Exception Handling with respect to method overriding?

There are certain restrictions while overriding a method in case of exception handling in Java. Broadly there are two rules –

- ❖ If super class method has not declared any exception using throws clause then subclass overridden method can't declare any checked exception though it can declare unchecked exception.
- ❖ If super class method has declared an exception using throws clause then subclass overridden method can do these things.
 -  Sub class method need not declare that exception.
 -  Sub class method can declare the same exception as declared in the super-class method.
 -  Sub class can declare the subtype exceptions of the exception declared in the super class method. But subclass method

cannot declare any super type of the exception declared in the super class method.

13. What is custom exception in java?

If you are creating your own exception that is known as custom exception or user-defined exception. Java custom exceptions are used to handle application specific exceptions.

With the help of custom exception, you can have your own custom messages.

Eg:

```
import java.util.Scanner;

class MarkException extends Exception {

    public MarkException(String s) {
        super(s);
    }
}

class MyMarks {
    private int mark;

    public void setMark(int mark) throws
MarkException {
        if (mark >= 0) {
            this.mark = mark;
            System.out.println("Good");
        } else {
            throw new MarkException("No negative
marks allowed");
        }
    }
}
```

```

        public int getMark() {
            return mark;
        }
    }

    public class ExamMark {
        public static void main(String[] args) {
            MyMarks mm = new MyMarks();
            try {
                Scanner sc = new Scanner(System.in);
                System.out.println("Please enter the
mark...");
                int mark = sc.nextInt();
                mm.setMark(mark);
            } catch (MarkException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}

```

14. What is ClassCastException in java?

ClassCastException is a RuntimeException which occurs when JVM is unable to cast an object of one type to another type.

15. Which class is the super class for all types of errors and exceptions in java?

java.lang.Throwable is the super class for all types of errors and exceptions in java.

16. What is the use of printStackTrace() method?

printStackTrace() method is used to print the detailed information about the exception occurred.

17. Give some examples of checked exceptions?

ClassNotFoundException, SQLException, IOException etc.

18. Give some examples of unchecked exceptions?

NullPointerException, ArrayIndexOutOfBoundsException,
NumberFormatException etc.

19. What is difference between final, finally and finalize in Java?

final and finally are keywords in java whereas finalize is a method.

a) final

-  final keyword can be used with variables so that they can't be reassigned.
-  final keyword can be used with methods to avoid overriding it in the subclasses.
-  final keyword can be used with class to avoid deriving other classes from that class.

b) finally

finally keyword is used with try-catch block to provide statements that will always get executed even if some exception arises or not; usually finally is used to write clean up codes.

c) finalize()

finalize() method is executed by Garbage Collector before the object is destroyed, it's a great way to make sure all the global resources are closed.

20. Can we have an empty catch block?

We can have an empty catch block but it's a bad practice. We should never have empty catch block because if the exception is caught by that block, we will have no information about the exception and it will be difficult to debug it. So there should be at least a logging statement to log the exception details in the console or in the log files.

21. Is it necessary that each try block must be followed by a catch block?

It is not necessary that each try block must be followed by a catch block. It should be followed by either a catch block or a finally block.

22. Can finally block be used without catch?

Yes, by try block with finally is possible.

23. What is Error in java?

- ❖ Error is a subclass of Throwable in java.
- ❖ Error indicates some serious problems that our application should not try to catch in java.

- ❖ Errors are abnormal conditions in application.
- ❖ Errors belong to unchecked type.

24. Is it allowed to use multiple catch block in java?

Yes.

When you have multiple catch blocks against a try, the order of catch blocks must be from most specific to general ones. i.e sub class exceptions must come first and then super classes later.

25. What are the differences between throw and throws?

throw keyword is used to throw an exception explicitly in java.	throws keyword is used to declare an exception in java.
throw is used inside the method. Example in java: <pre>public void m(){ if(expression) throw new MyException(); }</pre>	throws is used in method declaration. Example in java: <pre>public void m() throws FileNotFoundException{ }</pre>

throw is always followed by instance of an Exception class in java. Example: throw new MyException ()	throws is always followed by name of Exception class in java. Example: throws FileNotFoundException
throw can be used to throw only one exception at a time. Example: throw new MyException ()	throws can be used to declare multiple exceptions at a time. Example: throws FileNotFoundException, ClassNotFoundException

26. When NullPointerException is thrown in java?

- ❖ NullPointerException is thrown in java when an instance method of a null object is accessed.
- ❖ Any attempt to access or modify the field of a null object throws NullPointerException in java.

27. What is NumberFormatException in java?

java.lang.NumberFormatException is thrown when program attempts to convert a string to numeric types, but that the string is not in the appropriate format.

Scenarios where NumberFormatException may be thrown in java
String "29" is in appropriate format to be converted into numeric type int. So, NumberFormatException will not be thrown.

But in the following program NumberFormatException will be thrown.

Eg:

```
public class NumberFormatExceptionExample {  
    public static void main(String args[]) {  
        String s = "29a";  
        int i = Integer.parseInt(s);  
        System.out.println(i);  
    }  
}
```

28. When does an ArrayIndexOutOfBoundsException occurs?

In case of an array,

- ❖ If index accessed is Negative, ArrayIndexOutOfBoundsException will be thrown.
- ❖ If index accessed is equal to the size of array, ArrayIndexOutOfBoundsException is thrown

- ❖ If index accessed is greater than the size of array,
ArrayIndexOutOfBoundsException is thrown

29. What is IllegalMonitorStateException in java?

Before calling wait(), notify() and notifyAll() methods thread must own lock on object's monitor, means wait(), notify() and notifyAll() methods must be called either from synchronized blocks or synchronized method otherwise IllegalMonitorStateException will be thrown at runtime.

30. Is there any case where finally block will not be executed?

The finally block will not be executed i.e., program exits either by calling System.exit() or by causing an infinite loop.

31. Can we override a super class method which is throwing an unchecked exception with checked exception in the sub class?

No. If a super class method is throwing an unchecked exception, then it can be overridden in the sub class with same exception or any other unchecked exceptions but cannot be overridden with checked exceptions.

32. Can a catch or finally block throw exception in java?

Yes, catch or finally block can throw checked or unchecked exception but it must be handled accordingly.

33. Will the finally block be executed when the catch clause throws exception in Java?

Yes. finally block is executed even when an exception is thrown from anywhere in either try or catch block.

34. Can you catch OutOfMemoryError?

Yes, it can be handled as the same way as exception as it is a subclass of Throwable. However it is not a good idea to handle it.

35. What is OutOfMemoryError in java?

OutOfMemoryError is a sub class of java.lang.Error which occurs when JVM runs out of memory.

It is thrown when the Java Virtual Machine cannot allocate an object because it is out of memory, and no more memory could be made available by the garbage collector.

36. What is StackOverflowError in java?

StackOverflowError is a subclass of java.lang.Error which is thrown by the JVM when stack overflows.

37. What is rethrowing an exception?

Exceptions that raised in the try block are handled in the catch block. If it is unable to handle that exception, it can re-throw that exception using throw keyword. It is called re-throwing an exception.

38. Can we keep the statements after finally block if the control is returning from the finally block itself?

No, it gives unreachable code error. Because, control is returning from the finally block itself. Compiler will not see the statements after it. That's why it shows unreachable code error.

39. Explain try-with-resource in java?

The try-with-resources statement is a try statement that declares one or more resources. A resource is an object that must be closed after the program is finished with it. The try-with-resources statement ensures that each resource is closed at the end of the statement. Any object that implements `java.lang.AutoCloseable`, which includes all objects which implement `java.io.Closeable`, can be used as a resource.

```
static String readFirstLineFromFile(String path)
throws IOException {
    try (BufferedReader br =
        new BufferedReader(new
        FileReader(path))) {
        return br.readLine();
    }
}
```

When the try block finishes the `BufferedReader` and `FileReader` will be closed automatically. This is possible because these classes implement the Java interface `java.lang.AutoCloseable`.

40. Is finally block mandatory in Java?

finally is not mandatory.

The finally block always executes. This ensures that the finally block is executed even if an exception occurs or not. But finally is useful for more than just exception handling; it allows the programmer to avoid having cleanup code accidentally bypassed by a return, continue, or break.

INPUT/OUTPUT

1. What are the two types of I/O streams?

- ❖ Byte stream
- ❖ Character stream

2. What are the parent classes for I/O streams?

- ❖ Byte stream – InputStream & OutputStream
- ❖ Character stream – Reader & Writer

3. What is System.out.println()?

println() is a method of PrintStream class. out is a static object of PrintStream class defined in System class. System is a class belongs to java.lang package.

The facilities provided by the System class are standard input, standard output, and error output streams; access to externally defined properties and environment variables; a means of loading files and libraries; and a utility method for quickly copying a portion of an array.

4. Which class is used to read streams of characters from a file?

FileReader

5. Which class is used to write streams of characters to a file?

FileWriter

6. Which is the Parent class of FileInputStream ?

InputStream

7. Which exceptions should be handled with the following code ?

```
FileOutputStream fileOutputStream = new FileOutputStream(new  
File("newFile.txt"));
```

FileNotFoundException

8. What is Scanner class used for ? when was it introduced in Java ?

Scanner class introduced in Java 1.5 for reading data Stream from the input device.

9. What does the read() method of FileInputStream returns when it reaches the end of the file?

Return type of read() method of FileInputStream is int and it returns -1 when it reaches the end of the file.

10. What is the purpose of flush() method?

This method is used to ensure that total data should be written properly to the file including last character.

11. What are the advantages of buffered streams.?

The physical input and output involving the input-output devices are typically very slow when compared with the CPU processing speeds. So the buffered streams are used for various purposes. If buffered is not used, each read or write request is handled directly by the underlying OS. This can make a program much less efficient.

- ❖ Speed up the processing of input and output by reducing the number of reads and write instructions.
- ❖ Store the data as bytes for more efficient processing.

12. What is difference between using BufferedInputStream and FileInputStream?

- ❖ BufferedInputStream is buffered, but FileInputStream is not.
- ❖ A BufferedInputStream reads from another InputStream, but a FileInputStream reads from a file.
- ❖ BufferedInputStream is much faster as compared to FileInputStream.

13. What is serialization in java?

Object Serialization in Java is a process used to convert Object into a binary format which can be persisted into disk or sent over network to any other running Java virtual machine. The reverse process of creating object from binary stream is called deserialization in Java.

Two important methods:

`ObjectOutputStream.writeObject()` // serialize and write to file

`ObjectInputStream.readObject()` // read from file and deserialize

14. How to make a Java class Serializable?

Your Java class needs to implements `java.io.Serializable` interface and JVM will take care of serializing object in default format. This makes the object of that class eligible for serialization.

15. How many methods Serializable has? If no method then what is the purpose of Serializable interface?

Serializable interface exists in `java.io` package.

It doesn't have any methods and so known as a marker interface in Java.

When your class implements `java.io.Serializable` interface, an object of that class becomes eligible for serialization.

16. What is the need of Serialization?

The serialization is used :-

- ❖ To send state of one or more object's state over the network through a socket.
- ❖ To save/persist the state of an object in a file.

An object's state needs to be manipulated as a stream of bytes.

17. While serializing you want some of the members not to be serialized? How do you achieve it?

Declare the variable/state as transient and it will not be included during Java serialization process.

18. What will happen if one of the members in the class doesn't implement Serializable interface?

If you try to serialize an object of a class which implements `Serializable`, but the object includes a reference to a non- `Serializable` class then a `NotSerializableException` will be thrown at runtime.

19. Suppose you have a class which you serialized it and stored in persistence and later modified that class to add a new field. What will happen if you deserialize the object already serialized?

It depends on whether the class has its own `serialVersionUID` or not. If we don't provide `serialVersionUID` in our class, then the serialization runtime

will calculate a serialVersionUID for that class based on certain aspects of the class.

In such case deserialization will result in java.io.InvalidClassException.

So it is recommended that all serializable classes explicitly declare serialVersionUID.

Note: There are some incompatible changes for which you will get InvalidClassException even if you provide serialVersionUID explicitly.

20. Do we need to implement any method of Serializable interface to make an object serializable?

No. Serializable is a Marker Interface. It does not have any methods.

21. Are the static variables saved as the part of serialization?

No. The static variables belong to the class are not the part of the state of the object so they are not saved as the part of serialized object.

22. What is a transient variable?

These variables are not included in the process of serialization and are not the part of the object's serialized state.

23. What will be the value of transient variable after de-serialization?

It's default value.

e.g. if the transient variable is an int, it's value after deserialization will be zero.

24. What will happen if we have used List, Set and Map as member of class?

ArrayList, HashSet and HashMap implements Serializable interface, so if we will use them as member of class they will get serialized and deserialized as well.

25. Are primitive types part of serialization process?

Yes, primitive types are part of serialization process.

26. Why does serialization NOT save the value of static class attributes? Why static variables are not serialized?

The Java variables declared as static are not considered part of the state of an object since they are shared by all instances of that class. Saving static variables with each serialized object would have the following problems

- ❖ It will make a redundant copy of the same variable in multiple objects which makes it inefficient.
- ❖ The static variable can be modified by any object and a serialized copy would be stale or not in sync with the current value.

27. Can a serialized object be transmitted through network?

Yes, a Serialized object can be transmitted via the network as Java serialized object remains in form of bytes. Serialized objects can also be stored in disk or database as blob.

28. What are the compatible changes in Java Serialization?

- ❖ Adding a new field will not affect serialization process. The newly added field will be set to its default values when the object of an older version of the class is unmarshaled.
- ❖ Changes In access modifiers such as private, public, protected or default is compatible since they are not reflected in the serialized object stream.
- ❖ Changing a transient field to a non-transient field, static to non-static are compatible changes.
- ❖ Deleting an existing Serializable field.

- ❖ Changing non-transient field to transient, non-static to static.

29. What are the incompatible changes in Java Serialization?

- ❖ Changing the field type,
- ❖ Updating the class package.

30. How to give a serialVersionUID for a class?

You can give in your own number and make sure it is static and final.

Eg:

```
private static final long serialVersionUID = 1L;
```

MULTITHREADING

1. What is a thread in Java?

A thread is the entity within the process that can be scheduled for processing. A thread is a concurrent unit of execution. Or in other words you can say that it is part of a process that can run concurrently with other parts of the processes.

2. What is Multithreading?

The process of executing multiple threads simultaneously is known as multithreading. Java supports multithreading. The main advantage of multithreading is reducing CPU idle time and improving the CPU utilization. This makes the job to be completed in less time.

3. What are the benefits of multi-threaded programming?

In Multi-Threaded programming, multiple threads are executing concurrently that improves the performance because CPU is not idle incase some thread is waiting to get some resources. Multiple threads share the heap memory, so it's good to create multiple threads to execute some task rather than creating multiple processes. For example, Servlets are better in performance than CGI because Servlet support multi-threading but CGI doesn't.

4. What is the difference between thread and process?

- ❖ A program in execution is often referred as process. A thread is a subset(part) of the process.

- ❖ A process consists of multiple threads. A thread is a smallest part of the process that can execute concurrently with other parts(threads) of the process.
- ❖ A process is sometime referred as task. A thread is often referred as lightweight process.
- ❖ A process has its own address space. A thread uses the process's address space and share it with the other threads of that process.
- ❖ Threads have control over the other threads of the same process. A process does not have control over the sibling processes; it has control over its child processes only.

5. What is difference between user thread and daemon thread?

Any thread what we create from main thread is a user thread. By calling the `setDaemon(true)` method you can make a user thread as daemon thread.

JVM doesn't wait for any daemon thread to finish the Java program before exiting while JVM waits for all the non-daemon thread to complete before exiting the Java program.

When JVM terminates, it doesn't not invoke the daemon thread's finally block or unwind the stack. However it is not the case with non-daemon threads.

6. How to make a thread (user thread) to Daemon thread?

By calling `setDaemon(true)` method we can make a user thread to daemon thread.

Syntax:

```
thread.setDaemon(true);
```


7. How can we create a thread in Java?

We can create a thread by using any of the two following methods.

- 1) By implementing Runnable interface
- 2) By extending Thread class.

8. Differences between implementing Runnable interface and extending Thread class –

- ❖ Java does not support multiple inheritances of a class, but it allows to implement multiple interfaces at a time. Therefore, it will be better to implement Runnable than extending Thread class.
- ❖ Inheritance (Implementing Runnable is lightweight operation) : When we extend Thread unnecessary all Thread class features are inherited, but when we implement Runnable interface no extra feature are inherited, as Runnable only consists only of one abstract method i.e. run() method. So, implementing Runnable is lightweight operation.

9. Can we call run() method of a Thread class?

We can call run() method if we want but then it would behave just like a normal method and we would not be able to take the advantage of multithreading. In general run() method should be invoked by using start() method of a Thread class.

10. What do you understand by inter-thread communication?

- ❖ The process of communication between synchronized threads is termed as inter-thread communication.
- ❖ wait(), notify(), and notifyAll() are the methods used for inter thread communication.

11. How can we pause the execution of a Thread for specific time?

We can use Thread class's sleep() method to pause the execution of Thread for certain time. Note that this will stop the processing of thread for specific time, once the thread awake from sleep, it's state gets changed to runnable and based on thread scheduling, it gets executed.

12. What is synchronization?

Synchronization means allowing only one thread at a time to access an object.

Synchronization control the access the multiple threads to a shared resources. Without thread synchronization, one thread can modify a shared variable while another thread is working on the same variable, which leads to thread Interference.

13. What is the difference between notify() and notifyAll()?

notify() wakes up the one thread that called wait() on the same object, whereas the notifyAll() method wakes up all the waiting threads.

14. What does join() method do?

The join() method is used to hold the execution of currently running thread until the specified thread is dead(finished execution).

15. Can we start a thread twice in Java?

No, once a thread is dead, it can never be started again. Doing so will throw an IllegalStateException.

16. Is it important to acquire object lock before calling wait(), notify() and notifyAll()?

Yes, it's mandatory to acquire object lock before calling these methods on object. wait(), notify() and notifyAll() methods are always called from

synchronized block only, and before a thread enters into a synchronized block it has to acquire object lock . If we call these methods without acquiring object lock i.e. from outside synchronize block then java.lang. IllegalMonitorStateException is thrown at runtime.

17. What is life cycle stages of a thread?

- ❖ New
- ❖ Ready to run/Runnable
- ❖ Running
- ❖ Waiting/blocked/sleeping
- ❖ Terminated (Dead)

18. Explain the life cycle stages of a thread in detail?

- ❖ New : When instance of thread is created using new operator it is in new state, but the start() method has not been invoked on the thread yet, thread is not eligible to run yet.
- ❖ Runnable : When start() method is called on thread it enters into a runnable or ready to run state.
- ❖ Running : Thread scheduler selects thread to go from runnable to running state. In running state thread starts executing by entering run() method.
- ❖ Waiting/blocked/sleeping : In this state a thread is not eligible to run.
>Thread is still alive, but currently it's not eligible to run..
- ❖ By calling wait() method thread goes from running to waiting state. And it waits until it gets a notification from another thread.

- ❖ By calling `sleep()` method thread goes from running to sleeping state. In sleeping state it will wait for sleep time to get over.
- ❖ Terminated (Dead) : A thread is considered dead when it's `run()` method completes.

19. What is deadlock in multithreading?

Deadlock is a situation where two threads are waiting for each other to release lock held by them on resources.

20. How can Thread go from running to waiting state ?

- ❖ By calling `wait()` method thread goes from running to waiting state. In waiting state it will wait for other threads to release object monitor/lock. i.e it waits until it gets a notification from another thread.

21. How can thread return from waiting to runnable state ?

Once when the waiting thread gets a notification either by `notify()` or `notifyAll()` methods from another thread, monitor/lock becomes available and thread can again return to runnable state.

22. How can thread go from running to sleeping state ?

By calling `sleep()` method thread goes from running to sleeping state. In sleeping state it will wait for sleep time to get over.

23. What is significance of `sleep()` method?

- ❖ Definition : `sleep()` methods causes current thread to sleep for specified number of milliseconds (i.e. time passed in sleep method as parameter).
Ex- `Thread.sleep(10)` causes currently executing thread to sleep for 10 millisec.

- ❖ Thread state : when sleep() is called on thread it goes from running to sleeping state and can return to runnable state when sleep time is up.
- ❖ Exception : sleep() method must catch or throw compile time exception i.e. InterruptedException.
- ❖ Waiting time : sleep() method have got few options.
 - ✚ sleep(long millis) - Causes the currently executing thread to sleep for the specified number of milliseconds
 - ✚ sleep(long millis, int nanos) - Causes the currently executing thread to sleep for the specified number of milliseconds plus the specified number of nanoseconds.
- ❖ sleep() is a static method, causes the currently executing thread to sleep for the specified number of milliseconds.
- ❖ sleep() method belongs to java.lang.Thread class.
- ❖ Thread need not acquire object lock before calling sleep() method i.e. sleep() method can be called from outside synchronized block.

24. Difference between wait() and sleep() ?

- ❖ wait() should be called from synchronized block; wait() method is always called from synchronized block i.e. But sleep() method can be called from outside synchronized block i.e. sleep() method doesn't need any object monitor.
- ❖ wait() method belongs to java.lang.Object class but sleep() method belongs to java.lang.Thread class.
- ❖ wait() method is called on objects but sleep() method is called on Threads not objects.

- ❖ When wait() method is called on object, thread that held object's monitor goes from running to waiting state and can return to runnable state only when notify() or notifyAll() method is called on that object. And later thread scheduler schedules that thread to go from runnable to running state.
- ❖ when sleep() is called on thread it goes from running to sleeping state and can return to runnable state when sleep time is up.

25. What will happen if we don't override run method?

When we call start() method on thread, it internally calls run() method with newly created thread. So, if we don't override run() method newly created thread won't be called and nothing will happen.

26. What will happen if we override start method?

When we call start() method on thread, it internally calls run() method with newly created thread. So, if we override start() method, run() method will not be called when you call the start method.

27. Suppose you have 2 threads (Thread-1 and Thread-2) on same object.

Thread-1 is in synchronized method1(), can Thread-2 enter synchronized method2() at same time?

No, here when Thread-1 is in synchronized method1() it must be holding lock on object's monitor and will release lock on object's monitor only when it exits synchronized method1(). So, Thread-2 will have to wait for Thread-1 to release lock on object's monitor so that it could enter synchronized method2().

28. Suppose you have 2 threads (Thread-1 and Thread-2) on same object.

Thread-1 is in synchronized method1(), can Thread-2 enter static synchronized method2() at same time?

Yes, here when Thread-1 is in synchronized method1() it must be holding lock on object's monitor and Thread-2 can enter static synchronized method2() by acquiring lock on class.

29. Suppose you have a thread and it is in synchronized method and now can thread enter other synchronized method from that method?

Yes, here when thread is in synchronized method it must be holding lock on object's monitor and using that lock thread can enter other synchronized method.

30. Suppose you have thread and it is in static synchronized method and now can thread enter other static synchronized method from that method?

Yes, here when thread is in static synchronized method it must be holding lock on class and using that lock thread can enter other static synchronized method.

31. Suppose you have 2 threads (Thread-1 on object1 and Thread-2 on object2). Thread-1 is in synchronized method1(), can Thread-2 enter synchronized method2() at same time?

Yes, here when Thread-1 is in synchronized method1() it must be holding lock on object1's monitor. Thread-2 will acquire lock on object2's monitor and enter synchronized method2().

32. Suppose you have 2 threads (Thread-1 on object1 and Thread-2 on object2). Thread-1 is in static synchronized method1(), can Thread-2 enter static synchronized method2() at same time?

No. Here, when Thread-1 is in static synchronized method1() it must be holding lock on class and will release lock on class only when it exits static synchronized method1(). So, Thread-2 will have to wait for Thread-1 to release lock so that it could enter static synchronized method2().

33. What does join() method?

The join() method waits for a thread to die. In other words, it causes the currently running threads to stop executing until the thread it joins with completes its task. Join method is overloaded in Thread class in the following ways.

- ❖ public void join()throws InterruptedException
- ❖ public void join(long milliseconds)throws InterruptedException
- ❖ public void join(long milliseconds, int nanos) throws
InterruptedException

34. What is static synchronization?

If you make any static method as synchronized, the lock will be on the class not on the object. If we use the synchronized keyword before a non-static method so it will lock the object (one thread can access an object at a time) but if we use static synchronized, it will lock a class (one thread can access a class at a time).

35. What is Thread Scheduler in java?

In Java, when we create the threads, they are supervised with the help of a Thread Scheduler, which is the part of JVM. Thread scheduler is only responsible for deciding which thread should be executed.

Java thread scheduler also works for deciding the following for a thread:

- ❖ It selects the priority of the thread.
- ❖ It determines the waiting time for a thread
- ❖ It checks the nature of thread

36. Can a constructor be synchronized?

No, Constructor cannot be synchronized.

37. Is instance variable thread safe in java?

No, instance variable are stored in heap memory.

Multiple threads can access the same object.

You can run multiple threads in parallel.

They might be simultaneously modified by multiple threads calling unsynchronized instance methods.

38. Why are local variables thread safe in Java?

All local variables defined in your program will be allocated memory in the stack. So, When you create a thread it will have its own stack created. Two threads will have two stacks and one thread never shares its stack (that is, local variables) with other thread.

39. What is a Processes.?

Processes also known as programs or applications has its self-contained execution environment with its own memory space.

Process is considered to be heavyweight processes.

40. What is the default thread of every Java application?

"main" thread. This thread can create additional threads.

41. When does Thread Interference occur?

When two threads trying to act on the same data, latest thread wins and the first thread update is lost.

42. Does Thread class implements Runnable?

Yes.

43. What is the signature of Thread run() method?

```
public void run() {}
```

44. Does sleep() method throws any Exception?

Yes. We need to handle java.lang.InterruptedException.

45. Why do we call Thread.start() method which in turns calls run method?

run() method can be called directly however it will not be executing the method under new thread instead the method gets run on current thread.

When start() method is called it invokes run() by create a new Thread.

46. Give few example from Java API that throws InterruptedException.?

Thread.sleep(), Object.wait(), Thread.join() etc.

47. Is join() a overloaded method?

Yes.

❖ public final void join()

pause for the invoking thread object to complete.

❖ public final synchronized void join (long milliseconds)

pause for the invoking thread object to complete or resume after the specified milliseconds.

❖ public final synchronized void join (long milliseconds, int nano)

pause for the invoking thread object to complete or resume after the specified milliseconds plus nano.

48. Difference between thread state wait and blocked ?

A thread gets to wait state once it calls `wait()` on an Object. This is called Waiting State. Once a thread attains waiting state, it will continue to wait till some other thread `notify()` or `notifyAll()` on the object.

Once this thread is notified, it will not be runnable. It might be that other threads are also notified (using `notifyAll()`) or the first thread has not finished his work, so it is still blocked till it gets its chance. This is one of the scenarios of blocked State.

Once other threads have left and this thread gets chance, it moves to Runnable state after that it is eligible pick up work based on JVM threading mechanism and moves to run state.

49. What is Daemon thread in Java?

Daemon thread in Java are those thread which runs in background and mostly created by JVM for performing background task. A daemon thread is a thread that does not prevent the JVM from exiting when the Java program finishes but the thread is still running. An example for a daemon thread is the garbage collection.

The `setDaemon(boolean)` method can be used to change the Thread daemon properties before the thread starts.

50. Can a thread wait on multiple objects in Java?

No. A thread cannot wait on more than one object at a time.

The `wait()` and `notify()` methods are object specific and invoke on the object. The `wait()` method suspends the current thread of execution, and instructs the object to keep track of the suspended thread. The `notify()`

method tells the object to wake up the suspended threads that it is currently keeping track of.

51. Difference between synchronized and volatile keyword in Java.?

Volatile keyword is used on the variables and not on method while synchronized keyword is applied on methods and blocks not on variables.

Volatile does not acquire any lock on variable or object, but synchronized statement acquires lock on method or block in which it is used.

Volatile does not cause liveness problems such as deadlock while synchronized may cause as it acquires lock.

Volatile usually do not cause performance issues while synchronized block may cause performance issues.

52. Which is preferred - Synchronized method or Synchronized block?

Synchronized block is preferred as it provides more granular control. It only locks the critical section of the code and that eliminates unnecessary object locking.

53. Can we not override run method when we extend Thread class?

Yes. we can avoid overriding run method while extending Thread class. The start method will execute the default implementation which has no action.

54. Define Java thread pool.?

Java Thread pool is a group of worker threads that are waiting for the tasks to be assigned and be reused again. Worker threads return to the thread pool after it completes its task.

A group of fixed size threads are created in the thread pool. A thread from the thread pool is chosen and assigned a runnable task by the service provider. After completion of the job, thread is returns to the thread pool.

java.util.concurrent.Executors provide implementation of
java.util.concurrent.Executor interface to create the thread pool in java.

55. What are the advantages of using threadpool.?

Improves performance of multithreaded application as the thread pool eliminates the overhead of creating/recreating thread objects.

56. What is the priority for Daemon threads?

Priority of daemon threads is always 1, the lowest priority. Thread scheduler schedules these threads only when CPU is idle.

57. Explain yield() method of a thread.?

yield() method gives a notice to the thread scheduler that the current thread is willing to yield its current use of a processor. The thread scheduler is free to ignore this hint.

58. Can the Thread run method be overloaded?

Yes. You may overload the run method. However, the overloaded run method will be ignored by the Thread start method and it always starts the run method with no args.

59. What is the default priority of a thread? And what is the priority range?

Priority range is between 1 to 10. Default priority is 5.

60. Why sleep() method of Thread class is static ?

This is because when you call the sleep() method, you don't call the sleep() method of another thread. i.e sleep() is applicable for the current running thread. So you don't need to waste time on creating an object and call the sleep() method. So the sleep() method of the thread class is declared as static.

COLLECTION FRAMEWORK

1. What is Collection framework?

Collection is a framework, which is used to store and manipulate group of objects.

2. What are the basic operations of Collections?

- ❖ Add objects to the collection
- ❖ Remove objects from the collection
- ❖ Find out if an object (or group of objects) is in the collection
- ❖ Retrieve an object from the collection (without removing it)
- ❖ Iterate through the collection, looking at each element(object) one after another.

3. What are the basic interfaces of Java Collections Framework ?

Java Collections Framework provides a well designed set of interfaces and classes that support operations on a collection of objects. The most basic interfaces that reside in the Java Collections Framework are:

- ❖ Collection
- ❖ List
- ❖ Set
- ❖ Map
- ❖ Queue

4. What are the important classes implementing the List interface?

- ❖ ArrayList
- ❖ LinkedList
- ❖ Vector

5. What are the important classes implementing the Set interface?

- ❖ HashSet (not ordered,not sorted)
- ❖ LinkedHashSet(ordered,not sorted)
- ❖ TreeSet(sorted)

6. What are the important classes implementing the Map interface?

- ❖ HashMap(not ordered,not sorted)
- ❖ LinkedHashMap(ordered,not sorted)
- ❖ TreeMap(sorted, sorted based on key)

7. What is an Iterator ?

Iterator is an interface available in java.util package. It provides methods to iterate over any Collection.

8. What differences exist between Iterator and ListIterator ?

- ❖ An Iterator can be used to traverse the Set and List collections, while the ListIterator can be used to iterate only over List .
- ❖ By using Iterator we can retrieve the elements from Collection Object in forward direction only whereas ListIterator, allows you to traverse in either directions using hasPrevious() and previous() methods.

9. Difference between TreeSet and SortedSet?

SortedSet is an interface and TreeSet is a class. TreeSet implements SortedSet.

10. How to make a collection thread safe?

Use below methods:

- ❖ Collections.synchronizedList(list);
- ❖ Collections.synchronizedSet(set);
- ❖ Collections.synchronizedMap(map);

Above methods take collection as parameter and return same type of collection which are synchronized and thread safe.

11. What is the difference between Array and ArrayList?

- ❖ Array length is fixed where as ArrayList size grows automatically.
- ❖ Array can contain primitives and object data types where ArrayList can contain only object data types.
- ❖ Array size is provided by length property where as ArrayList size is provided by the size() method.
- ❖ Array has to be a specific type where as ArrayList can contain different types. However type safety can be achieved through generics if required in ArrayList.

12. What is the difference between List and Set?

- ❖ Set is unordered collection where List is ordered collection based on zero based index.
- ❖ List allow duplicate elements but Set does not allow duplicates.

13. What is the difference between ArrayList and LinkedList?

- ❖ ArrayList internally uses dynamic array to store the elements. LinkedList internally uses doubly linked list to store the elements.
- ❖ Manipulation with ArrayList is slow because it internally uses array. If any element is removed from the array, all the bits are shifted in memory. Manipulation with LinkedList is faster than ArrayList because it uses doubly linked list so no bit shifting is required in memory.
- ❖ ArrayList class can act as a list only because it implements List only. LinkedList class can act as a list and queue because it implements List and Deque interfaces.

- ❖ ArrayList is better for storing and accessing data purposes.
LinkedList is better for when manipulating of data is required.
- ❖ LinkedList can be traversed in reverse direction by using descending Iterator while there is no such method available for ArrayList to traverse in backward direction.
- ❖ By default, ArrayList creates an empty list with initial capacity of 10 (when capacity not specified) while LinkedList creates an empty list with no initial capacity.

14. What is the difference between HashSet and TreeSet?

- ❖ HashSet is not ordered while TreeSet is sorted.
- ❖ HashSet uses hashing algorithm and it is faster than TreeSet.
- ❖ HashSet can store null objects but storing null objects in TreeSet will result in NullPointerException.

15. What is the difference between Set and Map?

Set contains values only whereas Map contains both keys and values.

16. What is the difference between HashSet and HashMap?

- ❖ HashSet is an implementation of Set interface where as HashMap is an implementation of Map interface.
- ❖ HashSet contains only values where as HashMap contains keys and values.
- ❖ HashSet doesn't allow duplicates where as HashMap allows duplicate values, but no duplicate keys.
- ❖ HashSet uses add() method to insert where as HashMap uses put() method for insertion.

17. What is the difference between HashMap and TreeMap?

- ❖ HashMap is not ordered. TreeMap is sorted based on keys.
- ❖ HashMap can have one null key and multiple null values.
TreeMap cannot have null key but can have multiple null values.

18. What is the difference between Collection and Collections?

Collection is an interface whereas Collections is a class. Collection interface provides normal functionality of data structure to List, Set and Queue. But, Collections class contains many static utility methods for use with collections.

19. Difference between Vector and ArrayList?

- ❖ All the methods of Vector are synchronized. But, the methods of ArrayList are not synchronized.
- ❖ Vector is a Legacy class added in first release of JDK. ArrayList was part of JDK 1.2, when collection framework was introduced in java.
- ❖ By default, Vector doubles the size of its array when it is re-sized internally. But, ArrayList increases by half of its size when it is re-sized.

20. Difference between Comparable and Comparator?

Comparable and Comparator both are interfaces and can be used to sort collection elements.

1) Comparable : Class whose objects to be sorted must implement this interface and implement compareTo(Object o) method.

Comparator: Class whose objects to be sorted do not need to implement this interface. Instead a third class can implement this interface and implement compare(Object o1, Object o2) method. And the object of this third class can be used for sorting.

2) Comparable :Sorting logic must be in same class whose objects are being sorted. Hence this is called natural ordering of objects.

Comparator: Sorting logic is in separate class. Hence we can write different sorting based on different attributes of objects to be sorted.

3) Comparable is part of java.lang package.

Comparator is part of java.util package.

4) We can sort the list elements of Comparable type by Collections.sort(List) method.

We can sort the list elements of Comparator type by Collections.sort(List,Comparator) method.

21. What is the benefit of Generics in Collections Framework?

Java 1.5 came with Generics and all collection interfaces and implementations use it heavily. Generics allow us to provide the type of Object that a collection can contain, so if you try to add any element of other type it gives a compile time error.

This avoids ClassCastException at runtime because you will get the error at compilation itself. Also Generics make code clean since we don't need to use casting and instanceof operator.

22. What are different ways to iterate over a list?

You can iterate over a list using following ways:

- ❖ Iterator
- ❖ for loop
- ❖ for loop (Enhanced for loop)
- ❖ while loop

- ❖ do while loop
- ❖ Java 8 forEach() method etc.

23. How can we sort a list of Objects?

If we need to sort an array of Objects, we can use `Arrays.sort()`. If we need to sort a list of objects, we can use `Collections.sort()`. Both these classes have overloaded `sort()` methods for natural sorting (using `Comparable`) or sorting based on criteria (using `Comparator`).

`Collections` internally uses `Arrays` sorting method, so both of them have same performance except that `Collections` take some time to convert list to array.

24. How to convert an array of String to ArrayList?

```
String[] cities = {"Moscow", "Washington", "Brasilia", "Paris", "Warsaw"};
//Use Arrays utility class
List<String> cityList = Arrays.asList(cities);
```

25. How to convert ArrayList to Array?

The `toArray()` method of `ArrayList` is used to return an array containing all the elements in `ArrayList` in the correct order.

`toArray()` has overloaded method and are:

- a) **`public Object[] toArray();`**
- b) **`public <T> T[] toArray(T[] a);`**

a) Using toArray() method of ArrayList which has no arguments

The returned array is of type Object

```
import java.util.ArrayList;

public class ArrayListToArray {
    public static void main(String[] args) {
        ArrayList<String> petList = new
ArrayList<String>();
        petList.add("Cat");
        petList.add("Dog");
        petList.add("Parrot");
        Object petArray[] = petList.toArray();
        for(Object pet : petArray){
            System.out.println(pet);
        }
    }
}
```

b) Using toArray() method of ArrayList which takes type array as argument

The returned array is that of the specified array.

```
import java.util.ArrayList;

public class ArrayListToArray {
    public static void main(String[] args) {
        ArrayList<String> petList = new
ArrayList<String>();
        petList.add("Cat");
        petList.add("Dog");
        petList.add("Parrot");
        String[] petArray = petList.toArray(new
String[petList.size()]);
        for(String pet : petArray){
            System.out.println(pet);
        }
    }
}
```

26. How to reverse ArrayList?

The reverse method of Collections class can be used to reverse any ArrayList. It is a static method. Let's see the signature of reverse method:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;

public class ReverseAnArrayList {
    public static void main(String[] args) {
        ArrayList<String> nameList = new
ArrayList<String>();
        nameList.add("Ben");
        nameList.add("Henry");
        nameList.add("John");
        nameList.add("Williams");
        Collections.reverse(nameList);
        Iterator<String> it = nameList.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

27. How to remove duplicates from ArrayList?

This can be done in two ways.

1. Create a LinkedHashSet using the ArrayList which removes the duplicates. And then get back this set into an ArrayList.
2. Using the Stream API

```

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.stream.Collectors;

public class RemoveDuplicatesFromArrayList {

    public static void main(String[] args) {
        ArrayList<Integer> numberList=new
        ArrayList<Integer>();
        numberList.add(10);
        numberList.add(20);
        numberList.add(10);
        numberList.add(30);
        numberList.add(40);
        numberList.add(20);
        numberList.add(10);
        //Option 1: Creating a LinkedHashSet using the
        ArrayList.Thus duplicates
        //will be removed. And then get back this set
        into an ArrayList.
        LinkedHashSet<Integer> numbers = new
        LinkedHashSet<Integer>(numberList);
        ArrayList<Integer> numberListWithNoDuplicates
        =
            new ArrayList<Integer>(numbers);
        numberListWithNoDuplicates.forEach((num)-
        >System.out.println(num));
        System.out.println("-----");
        //Option 2: Using Stream API, new feature of Java 8

        List<Integer> numbersWithNoDuplicates =
        numberList.stream().distinct().collect(Collectors.toList
        ());
        numbersWithNoDuplicates.forEach((num)
        ->System.out.println(num));
    }
}

```


28. When to use ArrayList and LinkedList?

LinkedLists are better to use for the update operations whereas ArrayLists are better to use for the search operations.

29. What does get() method return if the key does not exist in HashMap?

null

30. How get() method will handle if two keys have the same hashCode?

In such cases a collision would have occurred while storing the elements using the put() method. That means same buckets with multiple entries. So the bucket location would have upgraded itself to store multiple entries by forming a linked data structure.

Now while fetching key.hashCode() would find the same bucket which will have multiple entries. So it will traverse through the entries to find the correct key using the equals() method. Once when it finds the matching key, the value from that entry will be returned.

31. Why Map interface does not extend Collection interface?

Some methods of Collection are incompatible with Map.

Map is a key/value pair(or a collection of entries) whereas Collection is a collection of objects stored in a structured manner and has a specified access mechanism.

32. What is the root interface in collection hierarchy ?

Root interface in collection hierarchy is Collection interface. Even though Collection interface extends Iterable interface, Iterable cannot be considered as the root interface of collection hierarchy as Iterable belongs to java.lang package not to java.util package

33. Which methods need to be overridden to use any object as a key in HashMap ?

To use any object as a key in HashMap, it needs to override equals() and hashCode() methods of Object class.

36. Which package contain all the classes which is related to Collection?

Java.util

37. What is hashCode?How do you get the hashCode?

hashCode is the integer representation of the memory address. hashCode() method of Object class return the hashCode of an object.

The Signature of the hashCode method is public int hashCode().

38. How HashMap works in java?

HashMap internally uses hashing algorithm for storing the elements. Every object has a hashCode. hashCode is nothing but it is an integer representation of the memory address. There is a method called

hashCode() which belongs to java.lang.Object class. When you call this method, you will get the hashCode of that particular object.

HashMap has a key value pair. The value is stored based on key. When you are adding an element in the HashMap using put() method, first it finds out the hashCode of the key. It may be a big number. So a rehashing takes place and finds a number between 0-15 . Because HashMap internally uses an array of size 16 for storing the elements. After calculating this number, the element is placed in that bucket as an entry.ie key value pair.

But there is a problem, when you are giving user defined object as key. Even if two objects are equal meaningfully their hashCode may differ. In that case it lands in the different buckets. So instead of replacing the earlier value it will be added in a different bucket. To overcome this problem, we must override both equals() and hashCode() methods in the class which is used as key.

Another issue is, two unequal objects may have the same hashCode. In that case both land in same bucket. This is called collision. But a call on equals() method will return false, so it forms a LinkedList and the new Entry will be added to the next node. Thereby you may have multiple entries in a single bucket.

While fetching a value using the get() method, it does the same thing. Calculates hashCode of key, does a rehashing and finds out the bucket. If that bucket has only one entry then it returns that value. And if that bucket has many entries then it will call the equals() method and match the entry's

The contract between `hashCode()` and `equals()` methods are “If two object are same, `hashCode` must be same. Two unequal objects may or may not have the same `hashCode()`).

39. How do you initialize an ArrayList?

In three ways,

- ❖ Using `add(Object obj)` method,

```
ArrayList<String> fruitList = new  
ArrayList<String>();  
fruitList.add("Apple");  
fruitList.add("Banana");  
fruitList.add("Orange ");
```

- ❖ Using Double brace initialization, (an anonymous inner class with an instance initializer.

```
ArrayList<String> fruitList = new ArrayList<String>()  
{  
    {  
        add("Apple");  
        add("Banana");  
        add("Orange");  
    }  
};
```

- ❖ Using `Arrays.asList()`,

```
List<String> fruitList =  
Arrays.asList("Apple", "Banana", "Orange");
```

40. What are the advantages of using Collections Framework?

- ❖ Collection classes provides a higher level of interface than arrays.

- ❖ Array size is fixed, where as collection classes like ArrayList, LinkedList etc are having flexibility in size.
- ❖ Allows efficient insertion at any location(eg: LinkedList).
- ❖ Collection framework has hash based implementing classes.
- ❖ Less programming/development effort.
- ❖ Improved code quality and well structured.
- ❖ Utility functions to perform processing/manipulation over the collection data eg: sort, search etc.
- ❖ Enhanced code readability.

41. What is Entry?

Map.Entry is an interface in Java collection framework which provides methods to access entry in the Map.

```
interface Map.Entry<K,V>
```

42. Name few collection classes that are synchronized or thread-safe ?

Vector, Stack, Hashtable etc. are some of the synchronized collection classes (or thread-safe).

43. How do you swap two elements in a list using Collections class API?

Using Collections.swap().

It swaps the elements at the specified positions in a mentioned list.

```
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class Swap {

    public static void main(String[] args) {
        List<String> nameList =
        Arrays.asList("Ben", "Adam", "John");
```

```

        System.out.println("Prints in the
insertion order");
        for (String name : nameList) {
            System.out.println(name);
        }

        // Swapping the priority between Ben and
Adam
        Collections.swap(nameList, 0, 1);

        System.out.println("Now the order of first
two elements changed");
        for (String name : nameList) {
            System.out.println(name);
        }
    }
}

```

44. Can a null element added to a TreeSet or HashSet?

One null is allowed in HashSet.

TreeSet uses the same concept as HashSet for internal logic, but uses NavigableMap for storing the elements.

NavigableMap is subtype of SortedMap which does not allow null keys. So essentially, TreeSet also does not support null keys. It will throw NullPointerException if you try to add null element in TreeSet.

45. How to design a good key for hashmap?

A good key object must provide same hashCode() again and again, no matter how many times it is fetched. Similarly, same keys must return true when compare with equals() method and different keys must return false.

For this reason, immutable classes are considered best candidate for HashMap keys.

46. Why ArrayList Is called dynamically growing array? How that dynamic behaviour Is achieved?

For ArrayList, data structure used for storing elements is array itself. When ArrayList is created it initializes an array with an initial capacity (default is array of length 10). When that limit is crossed another array is created which is 1.5 times the original array and the elements from the old array are copied to the new array. So we don't get any exception even if you add elements beyond the capacity of the created ArrayList. So generally ArrayList is called as growable array.

47. How to add elements To An Arraylist?

- ❖ List provides a method `add(E e)` which appends specified element to the end of the list. Using `add(E e)` method will mean keep adding elements sequentially to the list.
- ❖ Another add method - `add(int index, E element)` inserts the specified element at the specified position in this list.
- ❖ Third method `addAll(Collection<? extends E> c)` appends all of the elements in the specified collection to the end of this list.
- ❖ Fourth method `addAll(int index, Collection<? extends E> c)` inserts all of the elements in the specified collection into this list, starting at the specified position.

48. Does ArrayList allow null?

In ArrayList any number of null can be added.

49. Does Arraylist allow duplicate elements?

Yes. All List implementations allow duplicate elements.

50. How to join two or more array lists?

List provides addAll() method for achieving this.

- ❖ The addAll(Collection<? extends E> c) method appends all of the elements in the specified collection to the end of this list.
- ❖ The addAll(int index, Collection<? extends E> c) method inserts all of the elements in the specified collection into this list, starting at the specified position.

51. How to remove elements from an ArrayList?

ArrayList provides several methods to remove elements from the List. Since ArrayList internally uses array to store elements, one point to note is that when an element is removed from the List the remaining elements have to be shifted to fill the gap created in the underlying array.

- ❖ clear() - Removes all of the elements from this list.
- ❖ remove(int index) - Removes the element at the specified position in this list.
- ❖ remove(Object o) - Removes the first occurrence of the specified element from this list, if it is present.
- ❖ removeAll(Collection<?> c) - Removes from this list all of its elements that are contained in the specified collection.

52. What happens if Hashmap has duplicate keys?

If an attempt is made to add the same key twice, it won't cause any error but the value which is added later will override the previous value.

If you have a Map called cityMap and you add two values with same key in the cityMap, New York will override the Paris in the following case.


```
cityMap.put("5", " Paris ");  
cityMap.put("5", " New York ");
```

53. What is hash-collision in hash based collections?

In HashMap, using the key, a Hash is calculated and that hash value decides in which bucket the particular Map.Entry object will reside.

There may be situations where two unequal objects may have the same hash code. So a collision happens in this case. i.e. more than one keys having the same calculated hash value lands in the same bucket. In HashMap, in that case Entry objects are stored as a linked-list within the same bucket.

54. What is the hashCode() method?

hashCode() method is present in the java.lang.Object class. This method is used to get a unique integer value for a given object. We can see its use with hash based collections like HashTable or HashMap where hashCode() is used to find the correct bucket location where the particular (key, value) pair is stored.

56. When do we need to override hashCode() and equals() methods?

The default implementation of equals() method in the Object class is a simple reference equality check.

```
public boolean equals(Object obj){  
    return (this == obj);  
}
```

The default implementation of hashCode() in the Object class just returns integer value of the memory address of the object.

It becomes very important to override these two methods in case we are using a custom object as key in a hash based collection.

In that case we can't rely on the default implementation provided by the Object class and need to provide custom implementation of hashCode() and equals() method.

The contract between hashCode() and equals() methods are "If two object are same, hashCode must be same. Two unequal objects may or may not have the same hashCode().

57. Which Map implementation should be used if you want to retain the insertion order?

LinkedHashMap.

58. Which Map implementation should be used if you want map values to be sorted by keys?

TreeMap.

59. Which Set implementation should be used if you want the insertion order to be maintained?

LinkedHashSet.

61. What do you understand by fail-fast iterators?

Some iterators in java which are used for iterating over a collection immediately throw ConcurrentModificationException if there is a structural modification on the collection. i.e adding or removing an element when one thread is iterating over the collection. Such iterators are called fail-fast iterators.

Eg: Iterator on ArrayList, HashMap classes.

Some iterators do not throw any exceptions if a collection is structurally modified while iterating over it. The reason is they operate on a clone(copy) of the collection and not on the original collection. So they are not fail-fast iterators.

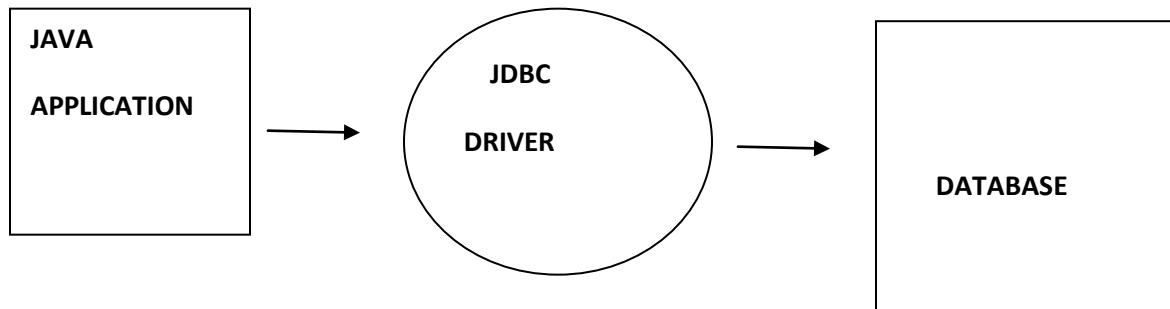
Ex: Iterator on CopyOnWriteArrayList, ConcurrentHashMap classes.

JDBC

1. What is JDBC?

Java JDBC is a Java API to connect and execute query with the database.

JDBC API uses jdbc driver to connect with the database.



2. What are the steps to connect to the database in java?

- ❖ Registering the driver class
- ❖ Creating connection
- ❖ Creating statement
- ❖ Executing queries
- ❖ Closing connection

3. What does the JDBC ResultSet interface?

The ResultSet object represents a row of a table. It can be used to change the cursor pointer and get the information from the database.

4. What is the difference between execute, executeQuery, executeUpdate in JDBC?

- ❖ execute(): it can be used for any kind of SQL Query.
- ❖ executeQuery() : it can be used for select query.
- ❖ executeUpdate(): it can be used to change/update table.

5. What is the return type of execute, executeQuery and executeUpdate?

- ❖ Return type of execute is boolean
- ❖ Return type of executeQuery is ResultSet object
- ❖ Return type of executeUpdate is int

6. Result Set's index Starts with 0 or 1?

Result Set's index starts with 1.

7. What is the role of Class.forName() while loading drivers?

Class.forName() loads, creates an instance of JDBC driver and register with DriverManager.

8. What is Connection object?

Connection interface consists of methods for interaction with the database.

9. What is a Statement?

Statement encapsulates an SQL statement which is passed to the database to be parsed, compiled, planned and executed.

10. How do you create a Connection object?

DriverManager.getConnection() method is used to create a Connection object.

getConnection(String url, String user, String password) Using a database URL with a username and password.

11. How do you insert images into Database using JDBC?

Images in the database using the BLOB data type wherein the image stored as a byte stream

12. What are the different types of JDBC Statements?

- ❖ Statement – regular SQL statement.
- ❖ PreparedStatement – more efficient than statement due to pre-compilation of SQL.
- ❖ CallableStatement – to call stored procedures on the database.

13. What is the use of blob, clob datatypes in JDBC?

These are used to store large amount of data into database like images, movie etc which are extremely large in size.

14. What is connection pooling?

Connection pooling is the mechanism by which we reuse the resource like connection objects which are needed to make connection with database. In this mechanism client doesn't required to make a new connection each time and then interact with database; instead connection objects are stored in connection pool and client can get it from there. so it's a best way to share a server resources among the clients and enhance the application performance.

15. What causes No suitable driver error?

"No suitable driver" occurs during a call to the DriverManager.getConnection method, may be for any of the following reasons:

- Due to failing to load the appropriate JDBC drivers before calling the getConnection method.
- It might be specifying an invalid JDBC URL, one that is not recognized by JDBC driver.

DESIGN PATTERNS

1. What is design pattern in Java?

Tested and proven solution for a common problem.

The benefit of design patterns lies in reusability and extensibility of the already developed applications.

2. Name few design patterns

- ❖ Singleton
- ❖ Factory
- ❖ Decorator
- ❖ Producer consumer
- ❖ Method Template
- ❖ Observer etc..

3. What are the different types of Java Design patterns.?

Java Design Patterns are divided into three categories:

- ❖ Creational (eg :Singleton)
- ❖ Structural (Eg: Decorator & proxy design pattern)
- ❖ Behavioral (Eg: observer)

Creational Patterns - These design patterns provide a way to create objects while hiding the creation logic, rather than instantiating objects directly using new keyword. This gives program more flexibility in deciding which objects need to be created for a given use case.

Structural Patterns - These design patterns concern class and object composition. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.

Behavioral Patterns - These design patterns are specifically concerned with communication between objects.

4. What is Singleton design pattern.?

Singleton pattern ensures that only one instance or object of a class exists with the application and make sure that instance is available globally so that there is no need to create new instance; of course there is no way to create more than one object.

5. How to implement a singleton pattern?

Create a public class having a private constructor. Private constructor ensures that the object cannot be created outside of the class.

Create a private static variable of the type Class itself.

Create a public static method e.g. getInstance() that returns the reference of the static variable, if the static variable is null then create a object for the class.

```
public class Singleton
{
    private static Singleton instance;

    private Singleton()
    {
    }

    synchronized public static Singleton getInstance()
    {
        if (instance == null)
        {
            instance = new Singleton();
        }
        return instance;
    }
}
```

```
}  
}
```

6. Why do we need Singleton class? (or) what are the advantages of using singleton pattern?

- ❖ Singleton class prevent creation of many objects for a class.
- ❖ Object creation of that class could be time-consuming.
- ❖ Objects that are used across the application be singleton as it is very convenient to maintain the state.

7. In how many ways can you create singleton pattern in Java?

Singleton objects can be created in two ways.

- ❖ Lazy loading
- ❖ Eager loading

8. What is eager initialization in singleton?

In eager initialization, the instance of Singleton Class is created at the time of class loading.

This is the easiest method but it has a drawback that instance will be created even if the client doesn't use it.

```
public class Singleton  
{  
    private static Singleton instance = new  
    Singleton();  
  
    private Singleton()  
    {  
    }  
  
    synchronized public static Singleton  
    getInstance()
```

```

    {
        return instance;
    }
}

```

9. What is Lazy Initialization in singleton?

Lazy initialization is a technique where we postpone the object creation until its first use.

```

public class Singleton
{
    private static Singleton instance;

    private Singleton()
    {
    }

    synchronized public static Singleton
getInstance()
    {
        if (instance == null)
        {
            instance = new Singleton();
        }
        return instance;
    }
}

```

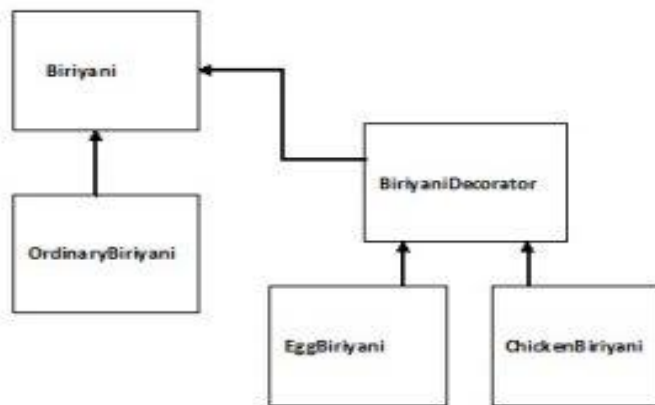
10. What is Decorator pattern in Java?

An object which is created functions the way it is designed. But it is possible to modify the functionality of an object at run time. Thanks to Decorator design pattern. It is a structural pattern and is a wrapper to an existing class.

Hope you remember this code which you have done while learning Java I/O:

```
FileInputStream fis = new FileInputStream("D:\\jThread\\jt.txt");  
BufferedInputStream bis = new BufferedInputStream(fis);
```

Let's take an interesting example:



www.jthread.com

So here I call Biryani as a component (An interface in this example) which has a method called `prepareBiryani()` and **OrdinaryBiryani** as a concrete component.

To decorate the biriyani I use a decorator called **BiryaniDecorator**. **EggBiryani** and **ChickenBiryani** are concrete decorators.

Now see the code for the same :

```

public interface Biriyani {
    public String prepareBiriyani();
}

public class OrdinaryBiriyani implements Biriyani {

    @Override
    public String prepareBiriyani() {
        return "Ordinary Biriyani";
    }
}

public abstract class BiriyaniDecorator implements
Biriyani {
    protected Biriyani biriyani;

    public BiriyaniDecorator(Biriyani biriyani) {
        this.biriyani = biriyani;
    }

    @Override
    public String prepareBiriyani() {
        return biriyani.prepareBiriyani();
    }
}

public class EggBiriyani extends BiriyaniDecorator {
    public EggBiriyani(Biriyani biriyani) {
        super(biriyani);
    }

    @Override
    public String prepareBiriyani() {
        return biriyani.prepareBiriyani() + addEgg();
    }

    private String addEgg(){
        return " :: Egg ";
    }
}

```

```
}  
}
```

```
public class ChickenBiriyani extends BiriyaniDecorator {  
    public ChickenBiriyani(Biriyani biriyani) {  
        super(biriyani);  
    }  
  
    @Override  
    public String prepareBiriyani() {  
        return biriyani.prepareBiriyani() + addChicken();  
    }  
  
    private String addChicken(){  
        return " :: Chicken ";  
    }  
}
```

All major coding is done. Now let's prepare and start serving:

```
public class TestBiriyani {  
  
    public static void main(String[] args) {  
  
        // Biriyani for vegetarians  
        Biriyani vegBiriyani = new OrdinaryBiriyani();  
        System.out.println(vegBiriyani.prepareBiriyani());  
        // It prints Ordinary Biriyani  
  
        // Biriyani for people who takes egg but chicken  
        Biriyani eggBiriyani = new EggBiriyani(new  
OrdinaryBiriyani());  
        System.out.println(eggBiriyani.prepareBiriyani());  
        // It prints Ordinary Biriyani :: Egg  
  
        // Biriyani for people who takes both egg and chicken  
        Biriyani chickenBiriyani = new ChickenBiriyani(new  
EggBiriyani(new OrdinaryBiriyani()));  
    }  
}
```

```

        System.out.println(chickenBiryani.prepareBiryani());
// It prints // Ordinary Biryani :: Egg :: Chicken
    }
}

```

11. Give an example of Decorator pattern in Java.?

BufferedReader and BufferedWriter are good example of decorator pattern in Java.

12. Explain Factory pattern in Java.?

Factory pattern is one of the Creational Design Patterns.

Factory pattern is used to construct objects such that they can be decoupled from the implementing system. It states that 'define an interface for creating an object, but let the subclasses decide which class to instantiate. The Factory method lets a class defer instantiation to subclasses'.

```

        public interface AdminManager {
            public void doSomething();
        }

        public class CorporateAdminManager implements
        AdminManager {
            public void doSomething() {
                System.out.println("do something");
            }
        }

        public class OfficeAdminManager implements AdminManager
        {
            public void doSomething(){
                System.out.println("do something
effectively");
            }
        }

        public class AdminManagerFactory {

```



```

    public static AdminManager getAdminManager(String s) {
        if ("o".equals(s)) {
            return new OfficeAdminManager();
        } else {
            return new CorporateAdminManager();
        }
    }
}

import java.util.Scanner;

public class Client1 {

    public static void main(String[] args) {
        System.out.println("Where are you now? o/c");
        Scanner sc=new Scanner(System.in);
        String s=sc.next();

        AdminManager
        am=AdminManagerFactory.getAdminManager();
        am.doSomething();
    }
}

```

13. When will you prefer to use a Factory Pattern?

The factory pattern is preferred in the following cases:

- ❖ A class does not know which class of objects it must create
- ❖ A factory pattern can be used where we need to create an object of any one of sub-classes depending on the data provided

14. What is proxy design pattern?

Proxy means not the original; i.e. an object representing a real object which provides the control for accessing the original object. It gives a protection for to the original object.

15. What is Observer design pattern?

The observer pattern is a behavioral pattern and it expresses the dependency between objects. i.e one object and many dependent objects in such a way that when the state of one object changes all the dependent objects get notified. The objects which are watching the state change of the another object are called observers and that is being watched is called Subject.

The listener in Java which get notified when an event happens on user interface control is a classic example of Observer pattern.

So let's take an example.

One day John and Abel decided to buy a watch online. But when searched for the same the specific model was not available. But there was an option in the online application to subscribe to get a notification when the respective stock arrives.

So below is the Subject interface which has methods for registering/removing observers and also to notify observers.

```
public interface Subject {  
    public void registerObserver(Observer observer);  
  
    public void removeObserver(Observer observer);  
  
    public void notifyObservers();  
}
```

And the Product class implements the Subject interface.

```
import java.util.ArrayList;
```

```

public class Product implements Subject {
    private ArrayList<Observer> observers = new
ArrayList<Observer>();
    private String productName;
    private String productType;
    private boolean availability;

    public Product(String productName, String
productType, boolean availability) {
        super();
        this.productName = productName;
        this.productType = productType;
        this.availability = availability;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getProductType() {
        return productType;
    }

    public void setProductType(String productType) {
        this.productType = productType;
    }

    public boolean isAvailability() {
        return availability;
    }

    public void setAvailability(boolean availability) {
        this.availability = availability;
        notifyObservers();
    }
}

```

```

    public ArrayList<Observer> getObservers() {
        return observers;
    }

    public void setObservers(ArrayList<Observer>
observers) {
        this.observers = observers;
    }

    public void notifyObservers() {
        for (Observer ob : observers) {
            ob.update(this.availability);
        }
    }

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }
}

```

Following is the Observer interface having an update method.

```

public interface Observer {
    public void update(boolean availability);
}

```

Person class is an implementation for Observer.

```

public class Person implements Observer {
    String personName;

    public Person(String personName) {
        this.personName = personName;
    }
}

```

```

    }

    public String getPersonName() {
        return personName;
    }

    public void setPersonName(String personName) {
        this.personName = personName;
    }

    public void update(boolean availabiliy) {
        String status = (availabiliy) ? "available" : "not
        available";
        System.out.println("Hello " + personName + ",
        Product is now " + status);
    }
}

```

Now it's time to test it.

```

    public class ObserverPatternMain {
        public static void main(String[] args) {
            Person john = new Person("John");
            Person abel = new Person("Abel");
            Product faTrackWatch = new
            Product("fastrack", "Watch", false);
            faTrackWatch.registerObserver(john);
            faTrackWatch.registerObserver(abel);
            faTrackWatch.setAvailability(true);
        }
    }
}

```



Whenever I discuss Java interview questions and answers with my students, I have noticed a different level of confidence on their faces. And many students have told me that such discussions have helped them to answer the interview questions clearly and with much confidence whenever they go for any interviews. In fact that was the inspiration for writing this book. I have included basic, medium and advanced levels of questions and answers in this book. I'm sure that this book will help many people to achieve their goals.

I thank Almighty for giving me the blessings to write this book. I thank my parents and family members for their support and prayers for me. Thank you jThread, my colleagues, students and all my well wishers for your support. When I started my journey in jThread, I knew very little about this technology and I'm proud to say that the content of this book is only the things which I learned in jThread.

May God bless you all.

